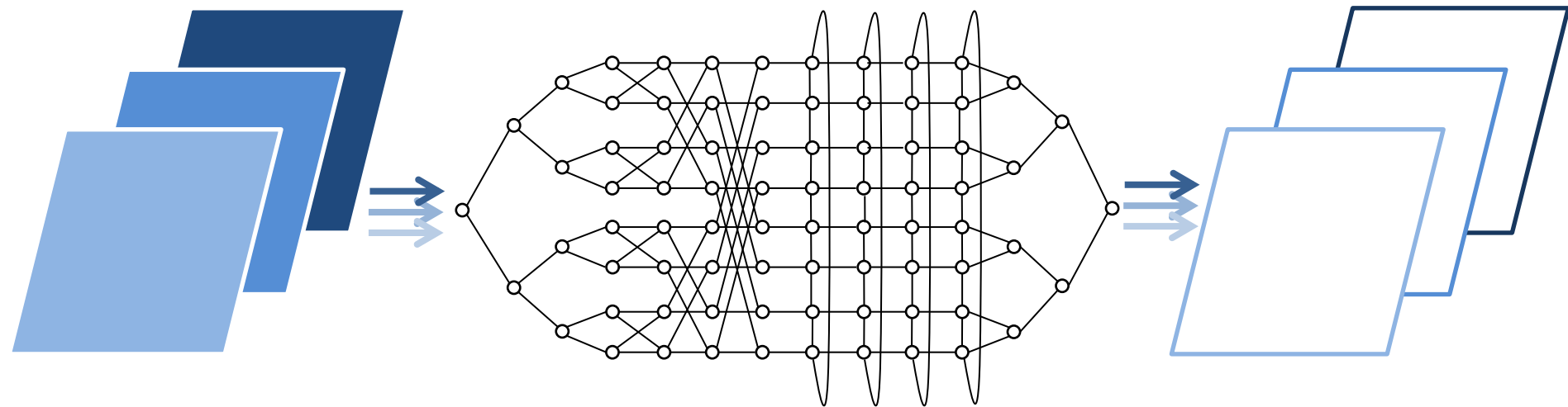
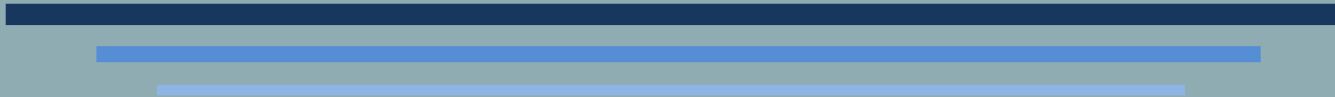


Programmation Parallèle

MPI: Message Passing Interface



WHAT YOU SHOULD CARE ABOUT FOR EFFICIENT MPI PROGRAMS



A small example of scalability



- Let's consider an MPI program
- Each rank should send a value to the next rank

```
If(rank==0)
```

```
    MPI_Ssend(&a, 1, MPI_INT, 1, ....)
```

```
    MPI_Recv(&b, 1, MPI_INT, P-1,...)
```

```
else
```

```
    MPI_Recv(&b, 1, MPI_INT, rank-1)
```

```
    MPI_Ssend(&a, 1, MPI_INT, (rank+1)%P,...)
```

A small example of scalability



Rank 0

Rank 1

Rank 2

Rank 3

Rank 4

A small example of scalability



Rank 0

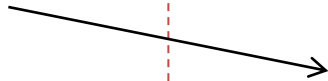
Rank 1

Rank 2

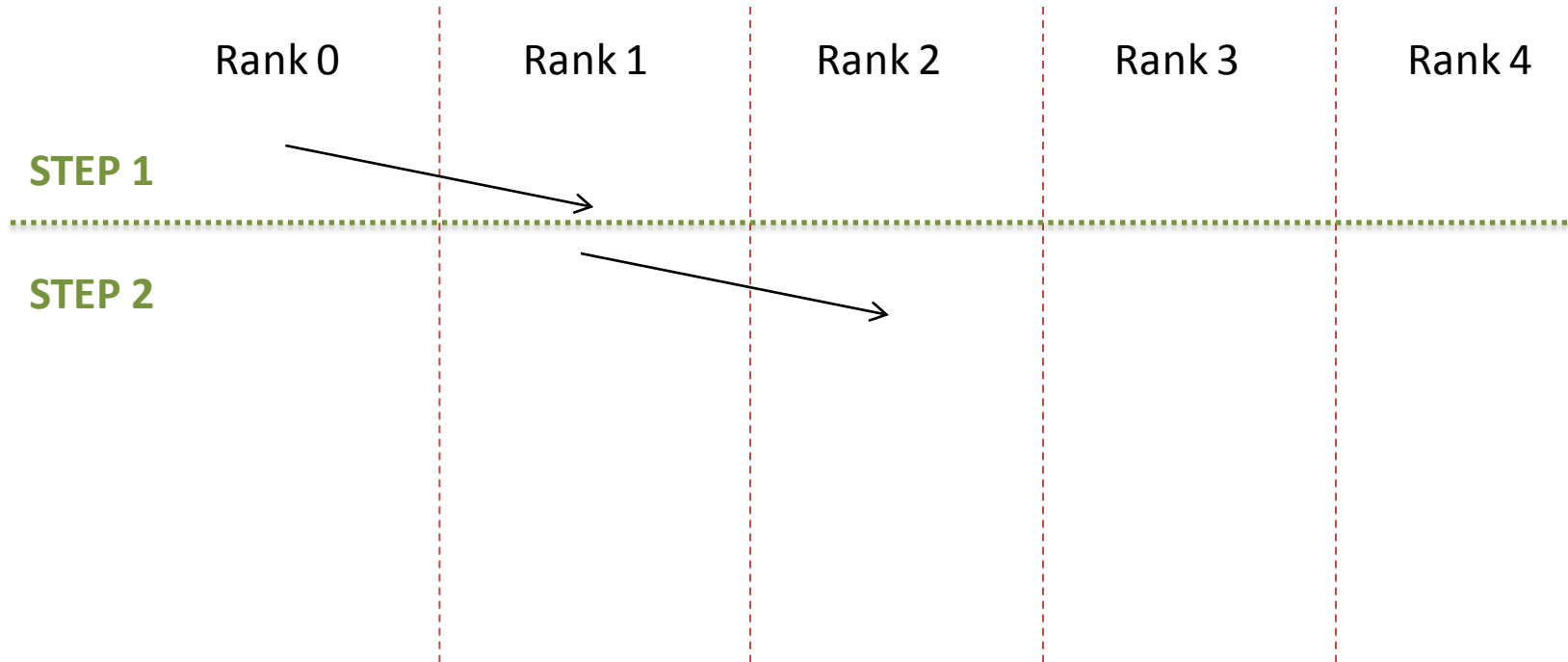
Rank 3

Rank 4

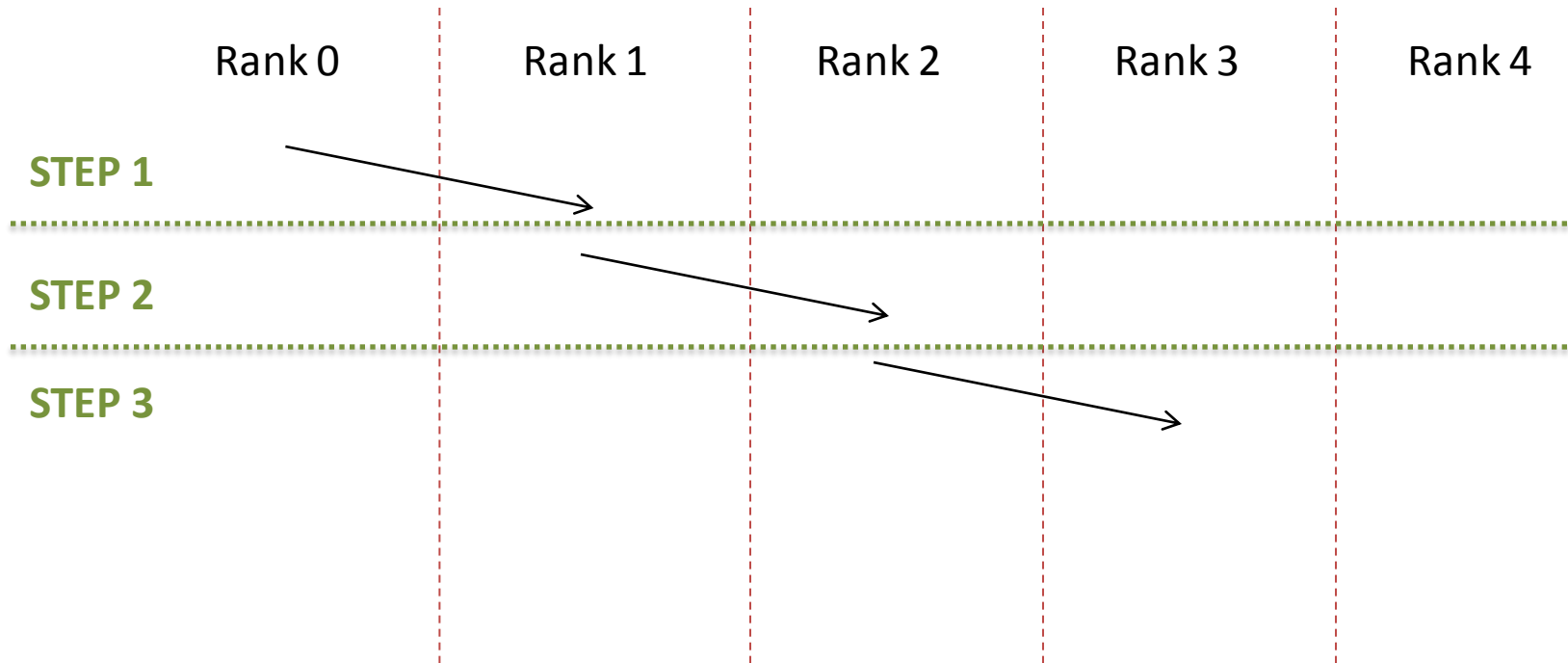
STEP 1



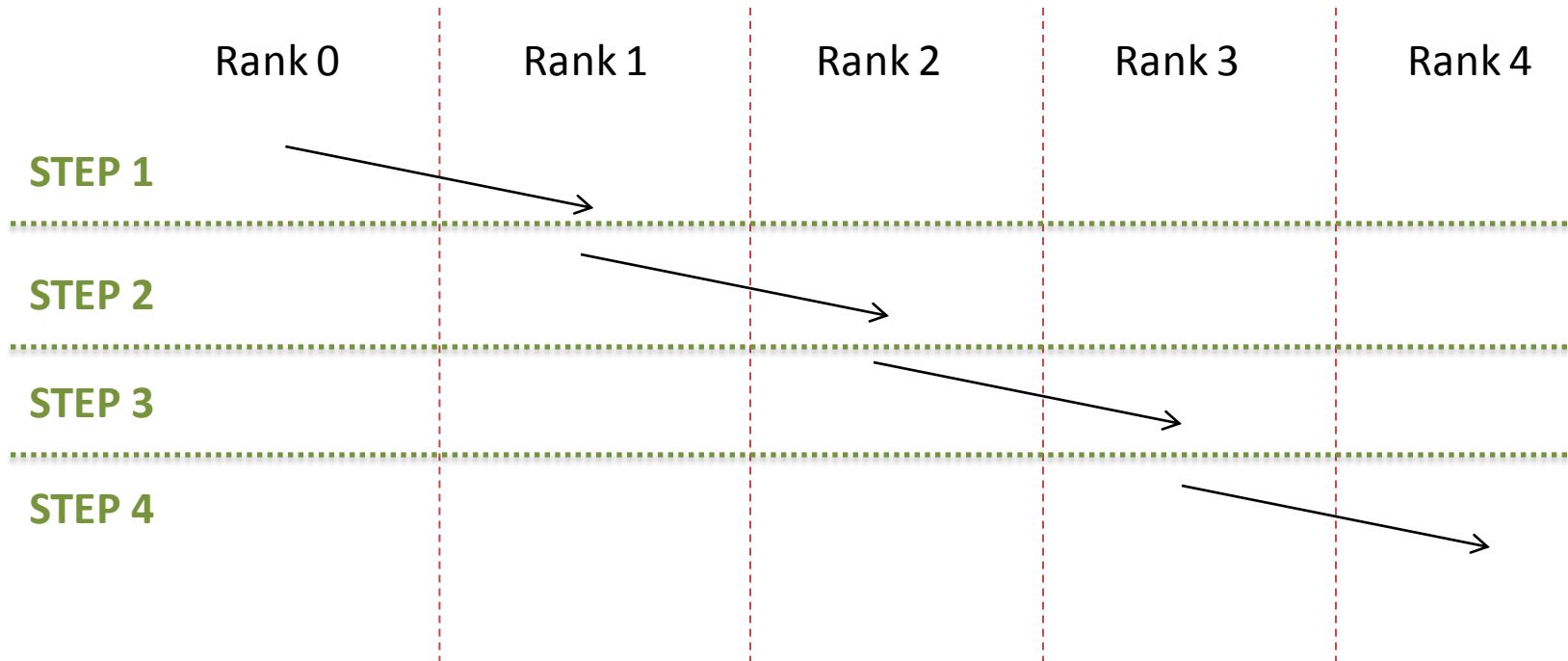
A small example of scalability



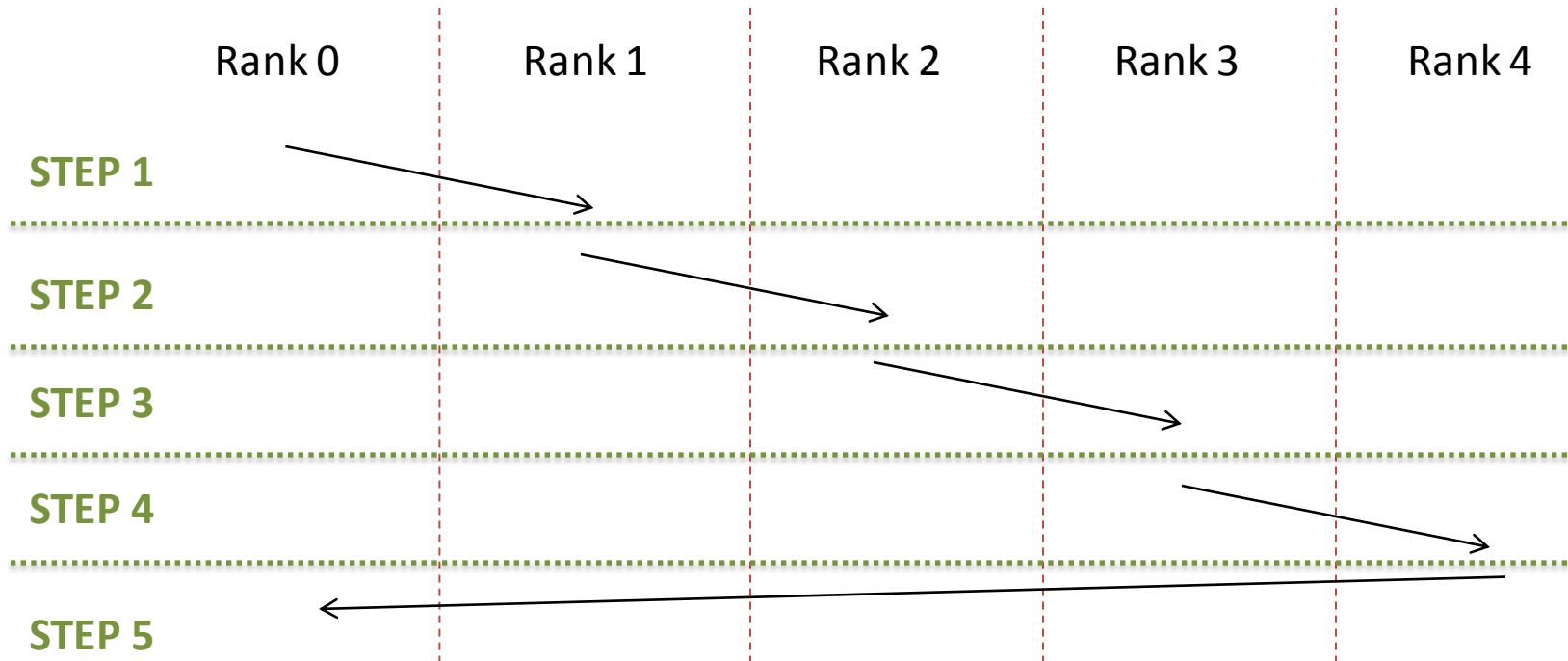
A small example of scalability



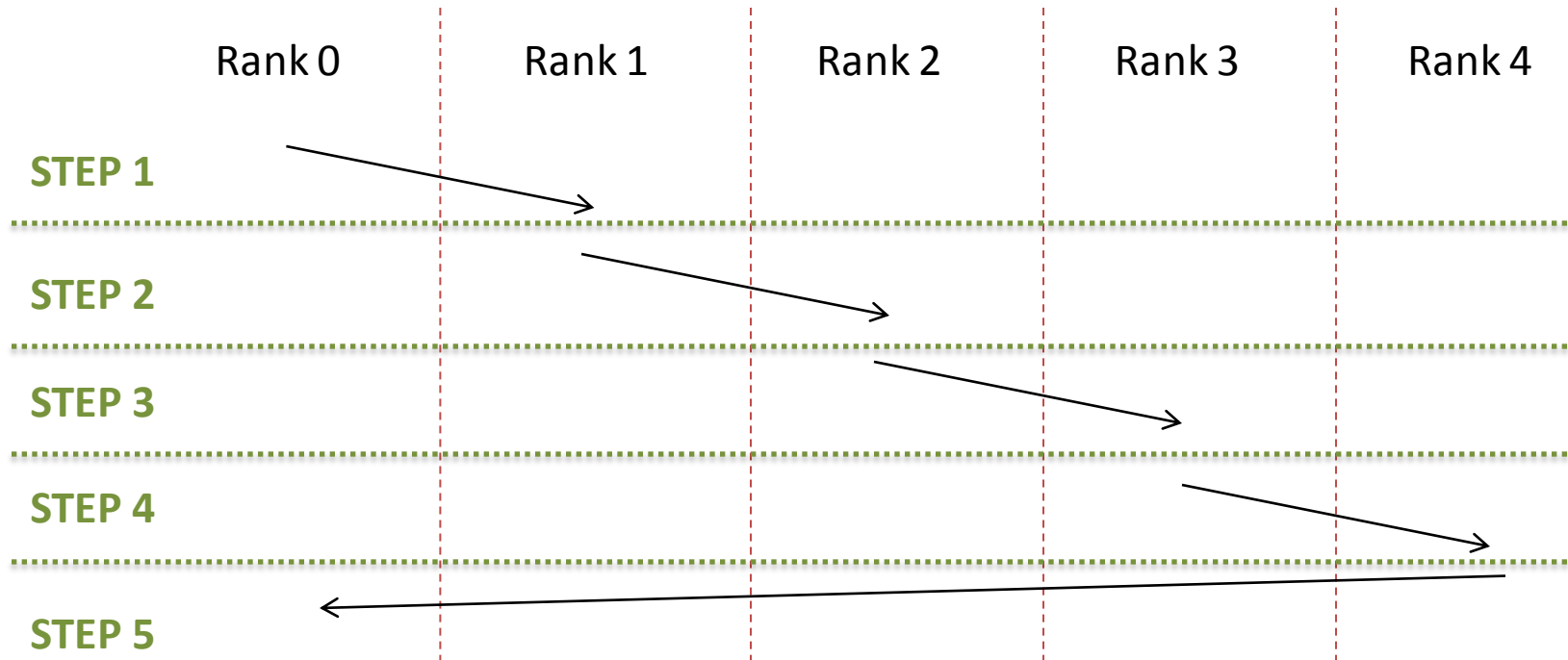
A small example of scalability



A small example of scalability



A small example of scalability



■ N steps

A small example of scalability



- Let's consider an MPI program
- Each rank should send a value to the next rank

```
if(rank==0)
```

```
    MPI_Ssend(&a, 1, MPI_INT, 1, ....)
```

```
    MPI_Recv(&b, 1, MPI_INT, P-1,...)
```

```
else
```

```
    MPI_Recv(&b, 1, MPI_INT, rank-1)
```

```
    MPI_Ssend(&a, 1, MPI_INT, (rank+1)%P,...)
```

A small example of scalability

- Let's consider an MPI program
- Each rank should send a value to the next rank

```
if(rank%2==0)
```

```
    MPI_Ssend(&a, 1, MPI_INT, 1, ....)
```

```
    MPI_Recv(&b, 1, MPI_INT, P-1,...)
```

```
else
```

```
    MPI_Recv(&b, 1, MPI_INT, rank-1)
```

```
    MPI_Ssend(&a, 1, MPI_INT, (rank+1)%P,...)
```

A small example of scalability



Rank 0

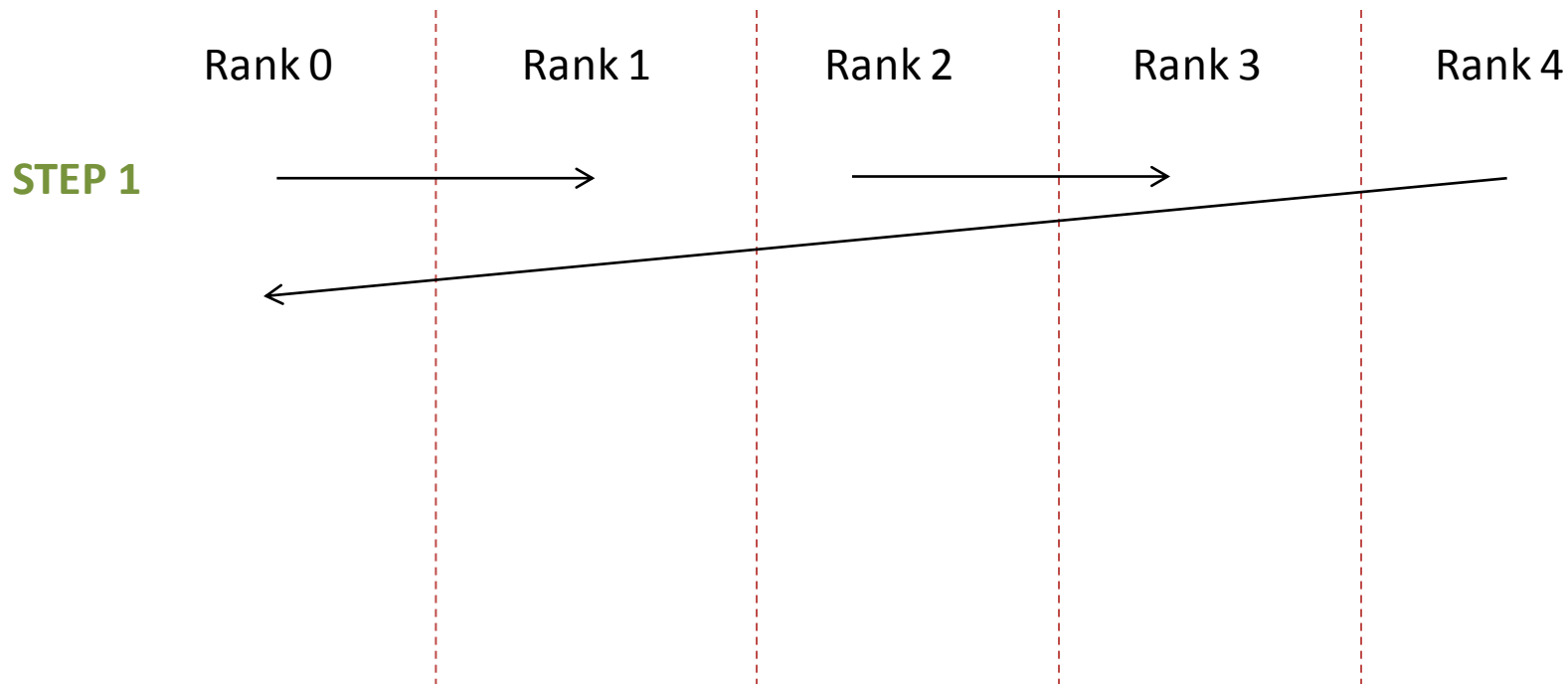
Rank 1

Rank 2

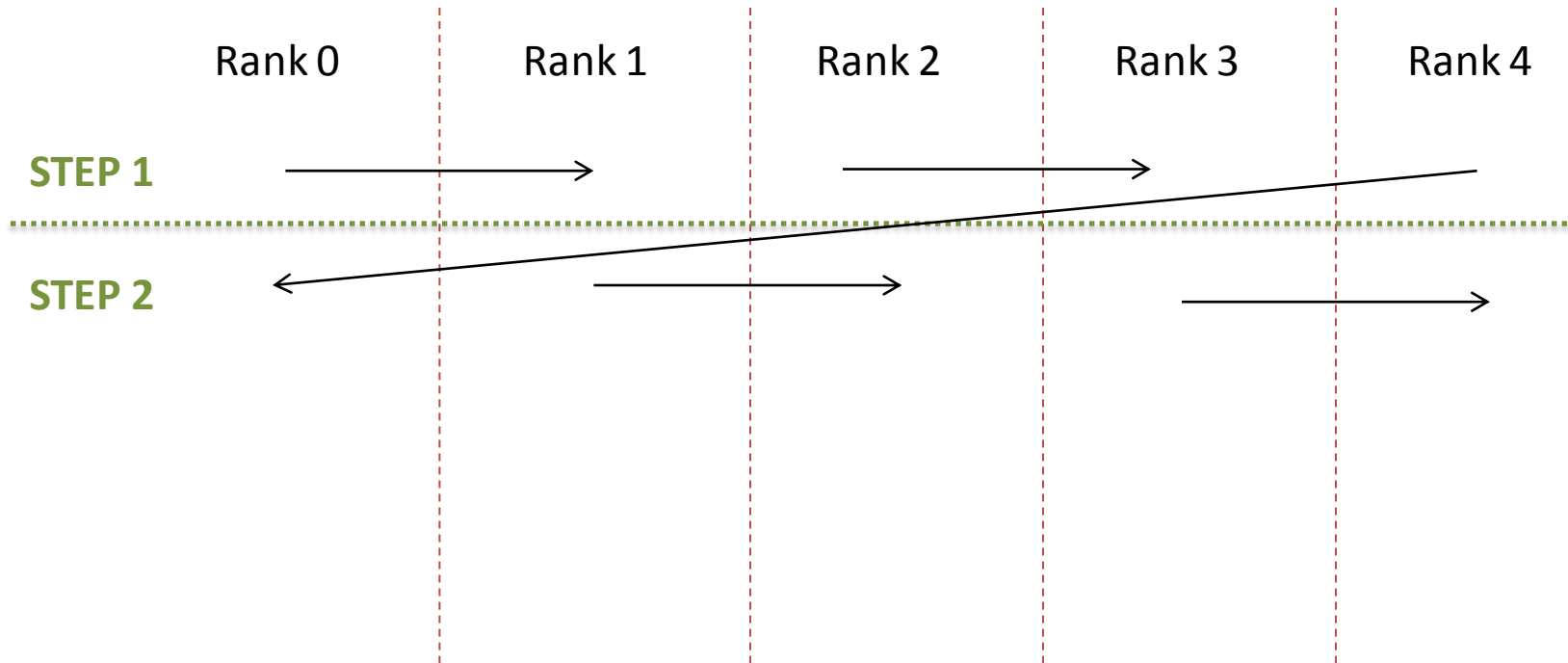
Rank 3

Rank 4

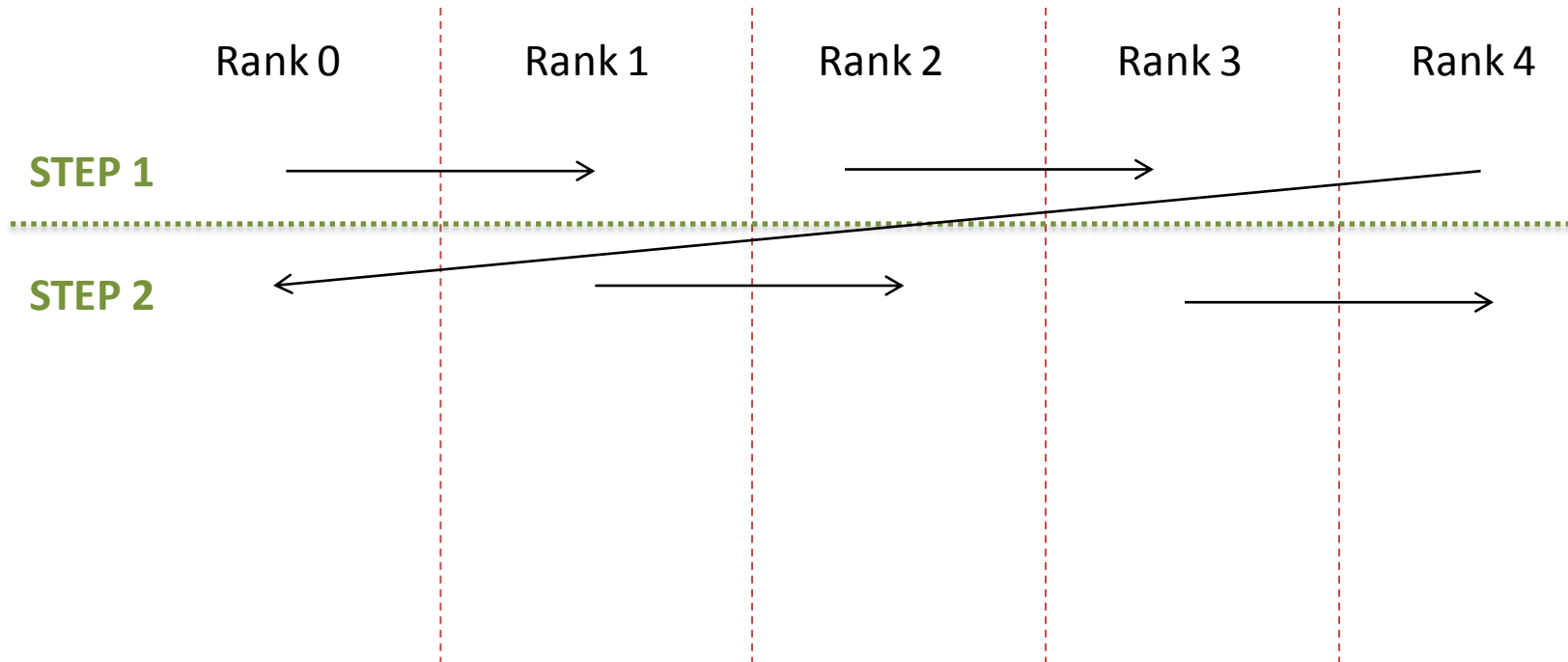
A small example of scalability



A small example of scalability



A small example of scalability



■ 2 steps

A small example of scalability

- Evaluation results

- 128 MPI processes on 128 nodes Intel Sandy Bridge w/ Infiniband

```
(c-inti)[jaegerj@r6login sr_ring_vs_modulo] $ ccc_mprun -N 128 -n 128 -p sandy ./main.pgr
RING      : min : 0.000083  and max : 0.008342
MODULO    : min : 0.000005  and max : 0.006315
```

- 24 MPI processes on 128 nodes Intel Skylake w/ Infiniband

```
(c-inti)[jaegerj@r6login sr_ring_vs_modulo] $ ccc_mprun -p sklb -N 24 -n 24 ./main.pgr
RING      : min : 0.000415  and max : 0.002164
MODULO    : min : 0.000004  and max : 0.000567
```

Know what you do



- A small difference in the code can have huge performance influence
- It is important to know the parallel algorithm/pattern of your communications
- A good scalability is a trade-off between number for communications and concurrency of these communications

Know what you will do



- The number of communications and the communication pattern may also depend on the code design
- It is important to know how to distribute the data among your MPI processes
 - Also depends on what will be send
 - And to which rank

Domain decomposition



- When running an MPI application, the computation is distributed among all the MPI processes
 - Hence, the same for the data needed by these computations
- In domain decomposition, a mesh is distributed among MPI processes
- Communications occur on the borders of these partitions
 - Halo cells or ghost cells

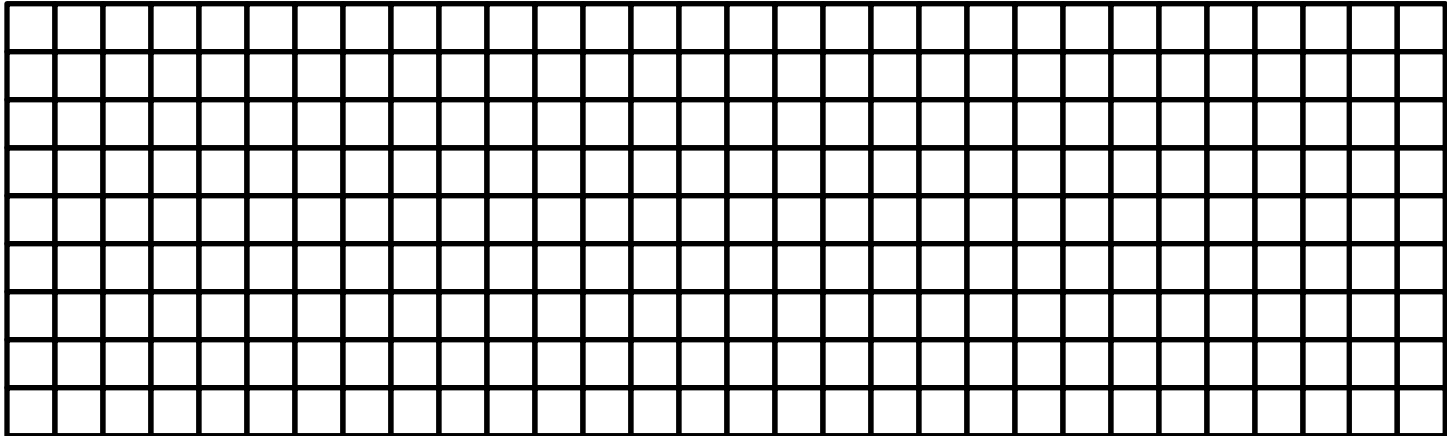
Domain decomposition



- Let's consider a very simple mesh: a matrix
- Computation: Matrices addition
- The matrices are decomposed in several parts
 - Each sub-computation is done on different MPI processes
- How to decompose our matrices?

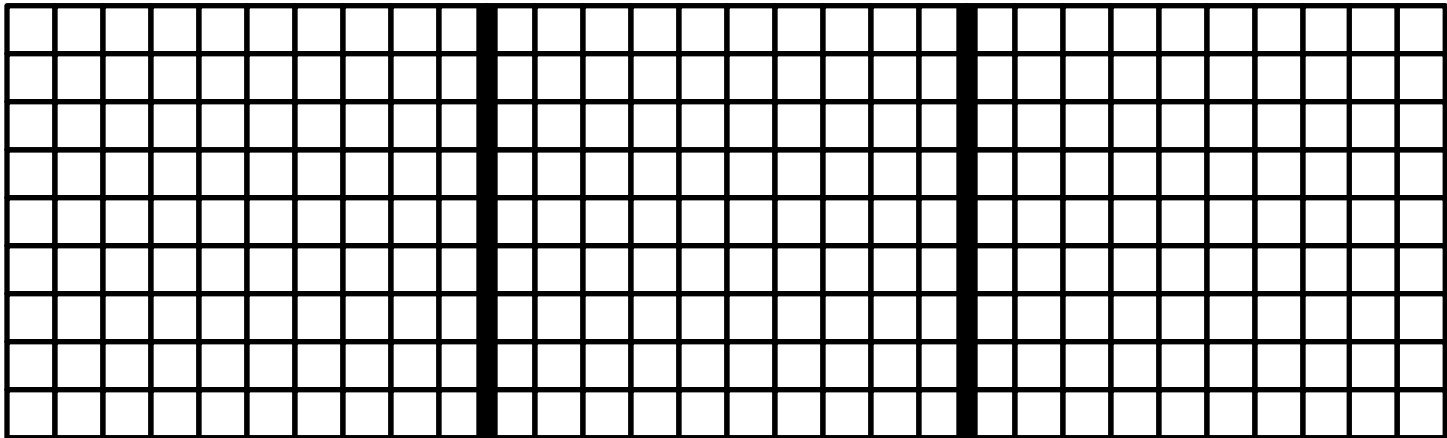
DD example: 2D matrix stencil

- a 30x9 integer Matrix
- 3 MPI processes
- Jacobi 4-point stencil



DD example: 2D matrix stencil

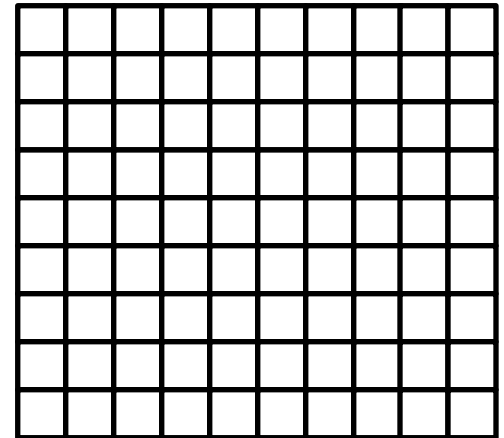
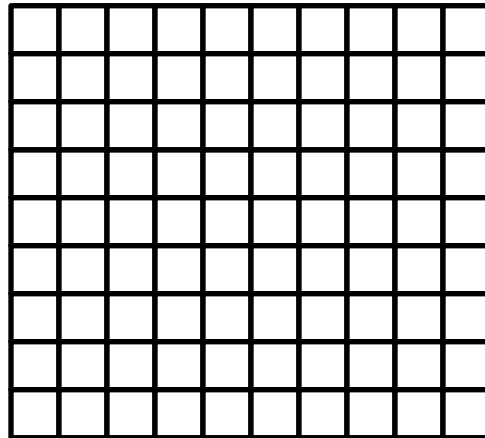
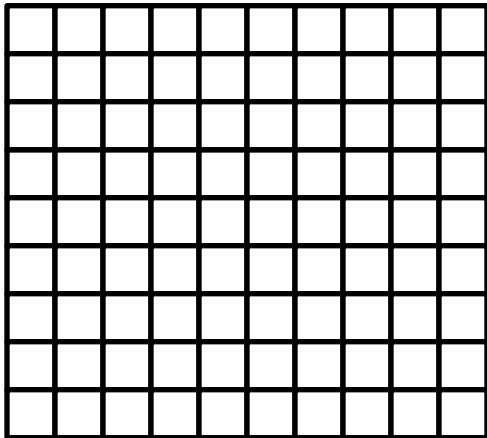
- a 30x9 integer Matrix
- 3 MPI processes
- Jacobi 4-point stencil
- Vertical decomposition



DD example: 2D matrix stencil



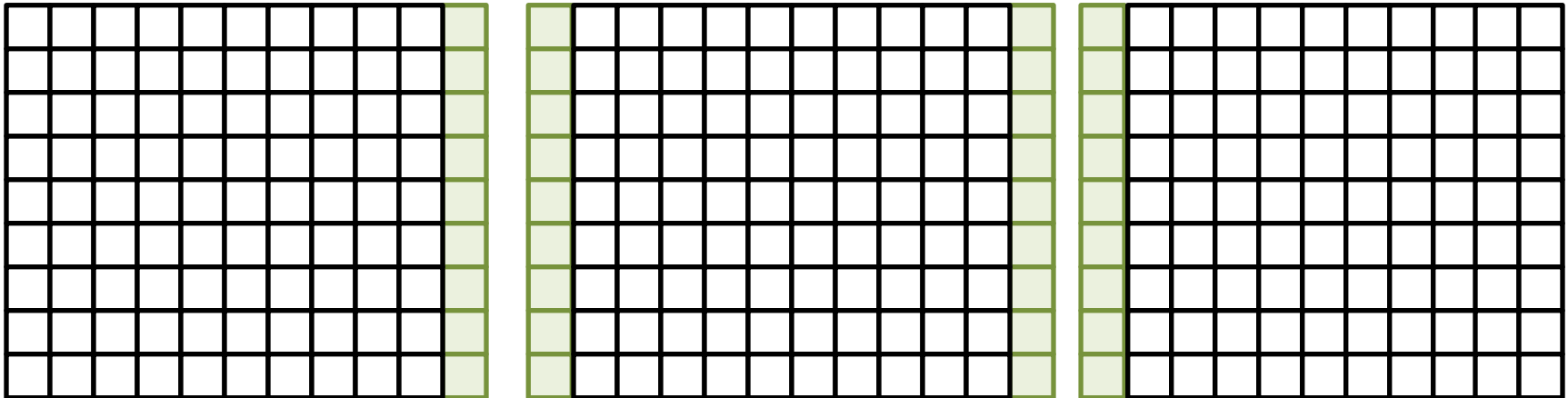
■ Vertical decomposition



DD example: 2D matrix stencil

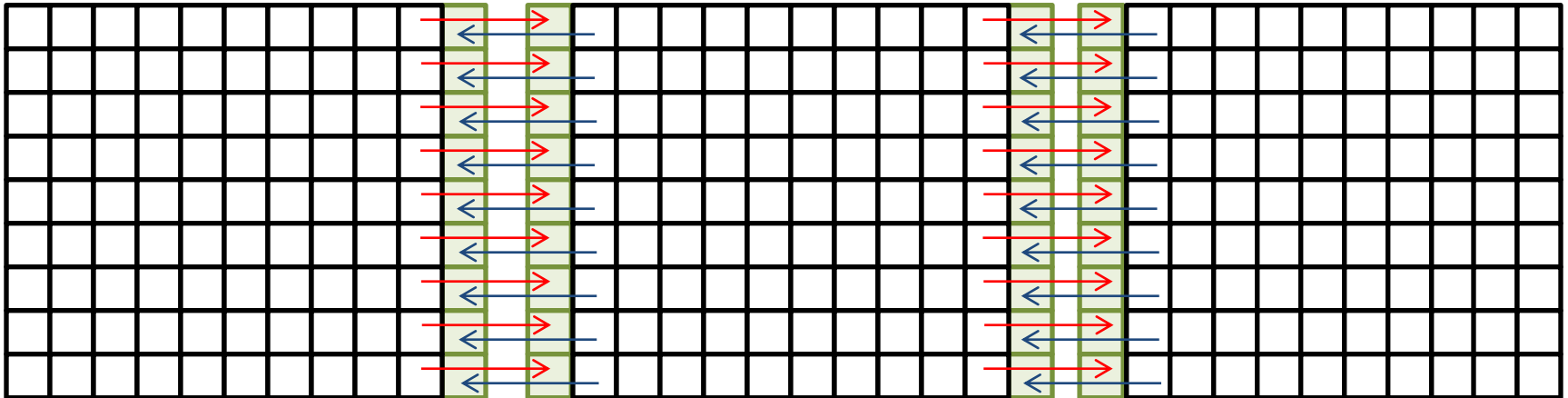


■ Vertical decomposition



DD example: 2D matrix stencil

- Vertical decomposition
- Ghost cells exchange:

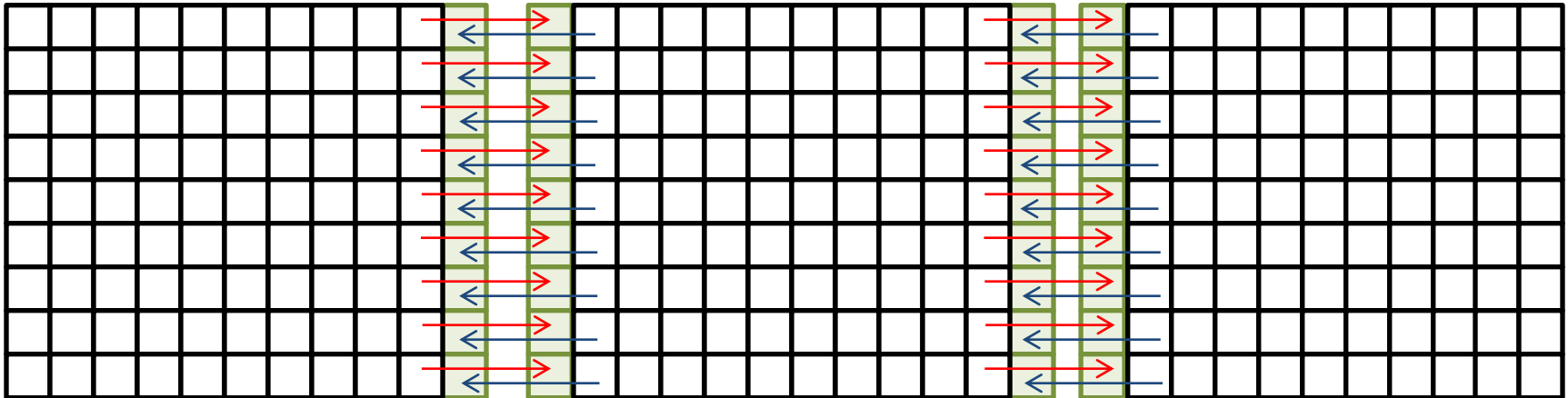


DD example: 2D matrix stencil

- Vertical decomposition

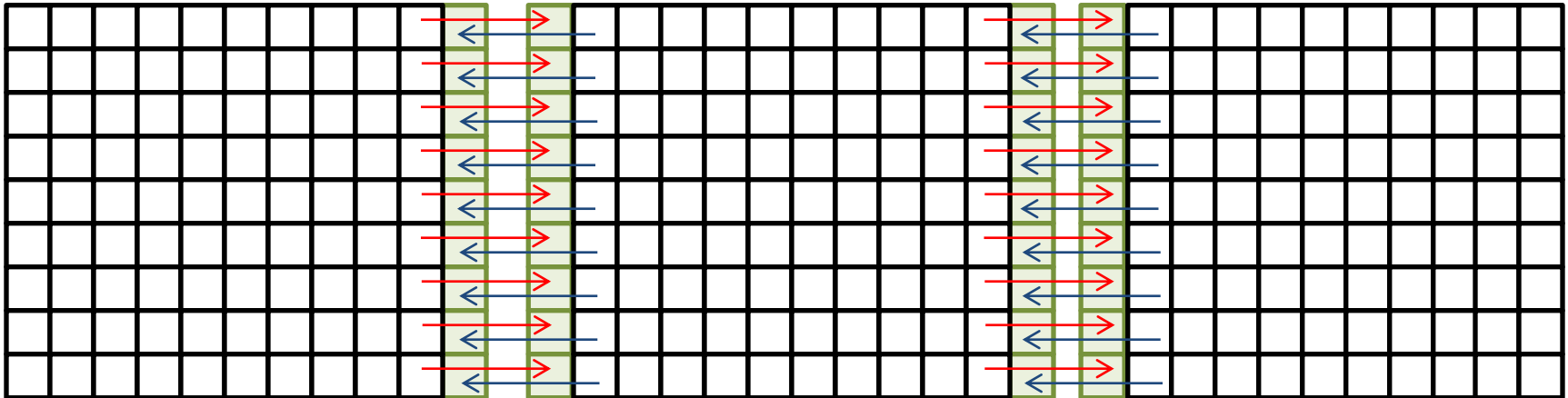
- Ghost cells exchange:

- `send(&(a[0][1], 1, MPI_INT, ...) send(&(a[1][1], 1, MPI_INT, ...)`
- `send(&(a[2][1], 1, MPI_INT, ...) send(&(a[3][1], 1, MPI_INT, ...)`
- ...
- `send(&(a[0][(L/3)], 1, MPI_INT, ...) send(&(a[1][(L/3)], 1, MPI_INT, ...)`



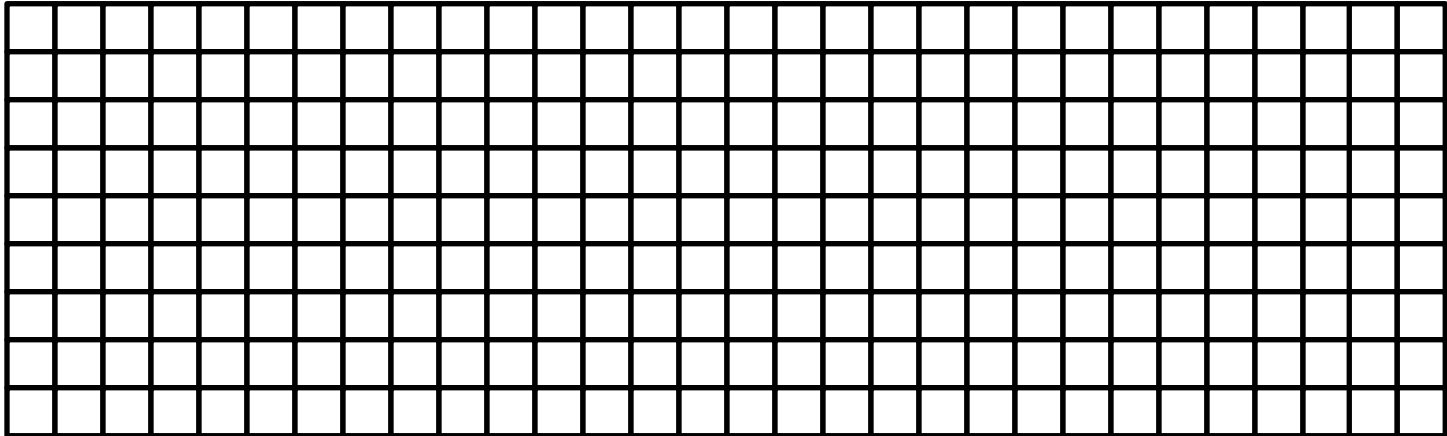
DD example: 2D matrix stencil

- Vertical decomposition
- Ghost cells exchange: **18 sends** (height)
 - `send(&(a[0][1], 1, MPI_INT, ...) send(&(a[1][1], 1, MPI_INT, ...)`
 - `send(&(a[2][1], 1, MPI_INT, ...) send(&(a[3][1], 1, MPI_INT, ...)`
 - ...
 - `send(&(a[0][(L/3)], 1, MPI_INT, ...) send(&(a[1][(L/3)], 1, MPI_INT, ...)`



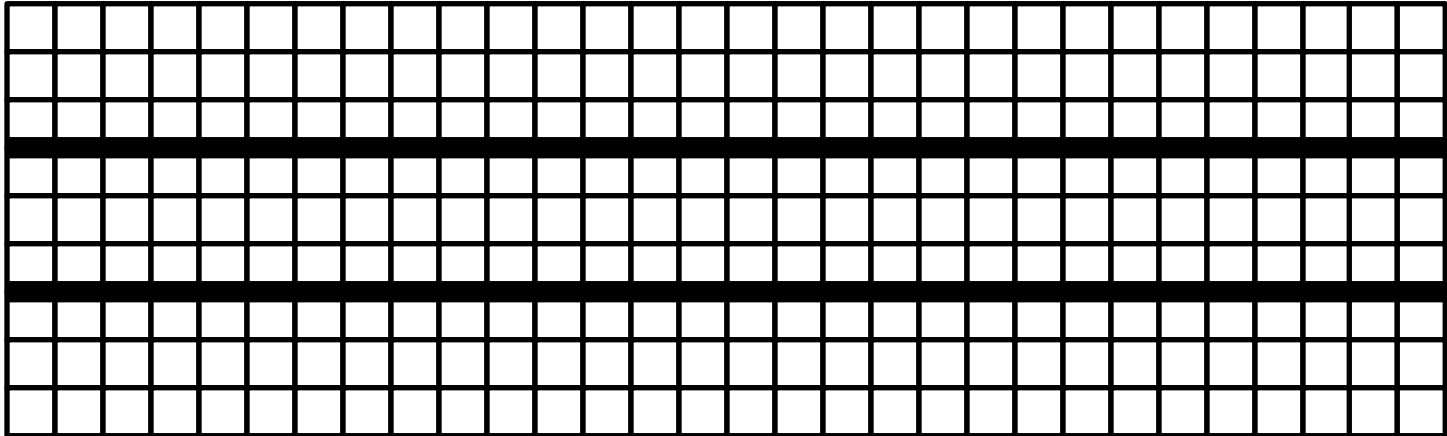
DD example: 2D matrix stencil

- a 30x9 integer Matrix
- 3 MPI processes
- Jacobi 4-point stencil



DD example: 2D matrix stencil

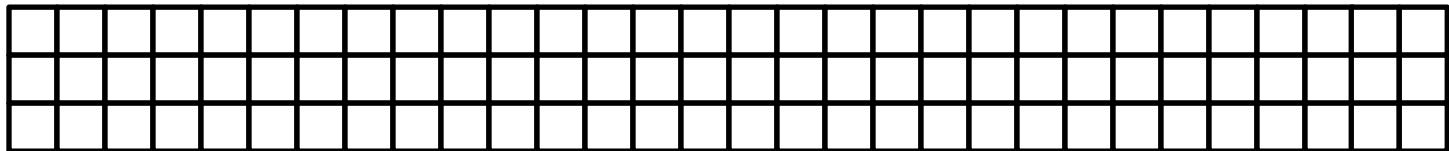
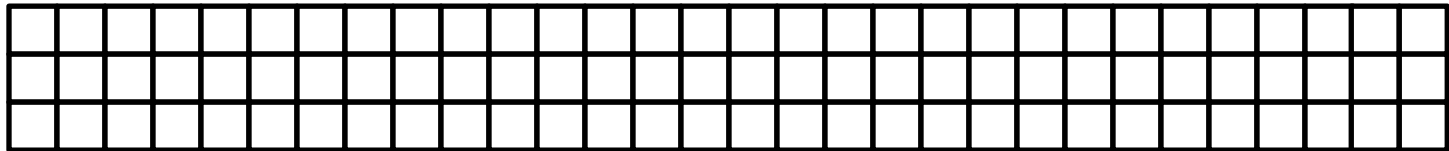
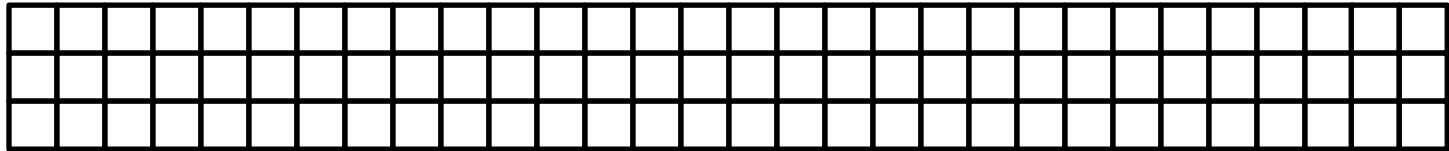
- a 30x9 integer Matrix
- 3 MPI processes
- Jacobi 4-point stencil
- Horizontal decomposition



DD example: 2D matrix stencil



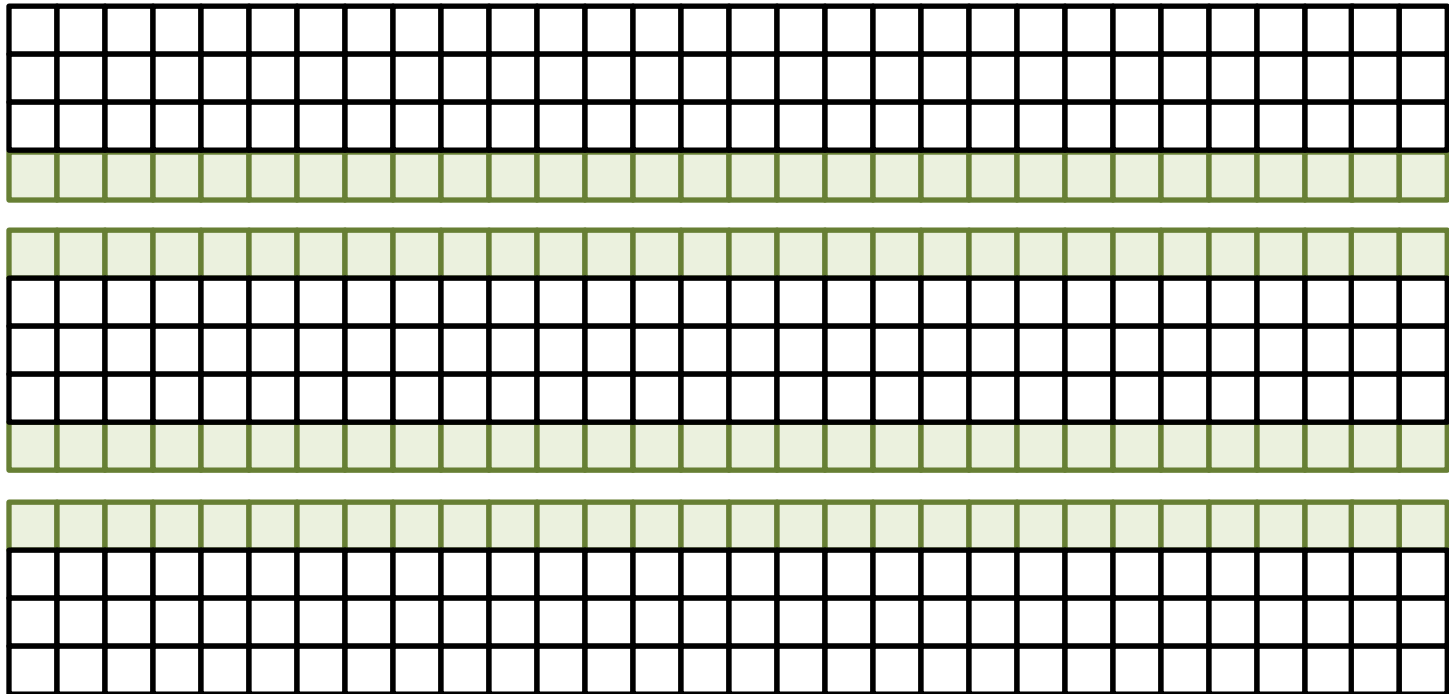
- Horizontal decomposition



DD example: 2D matrix stencil

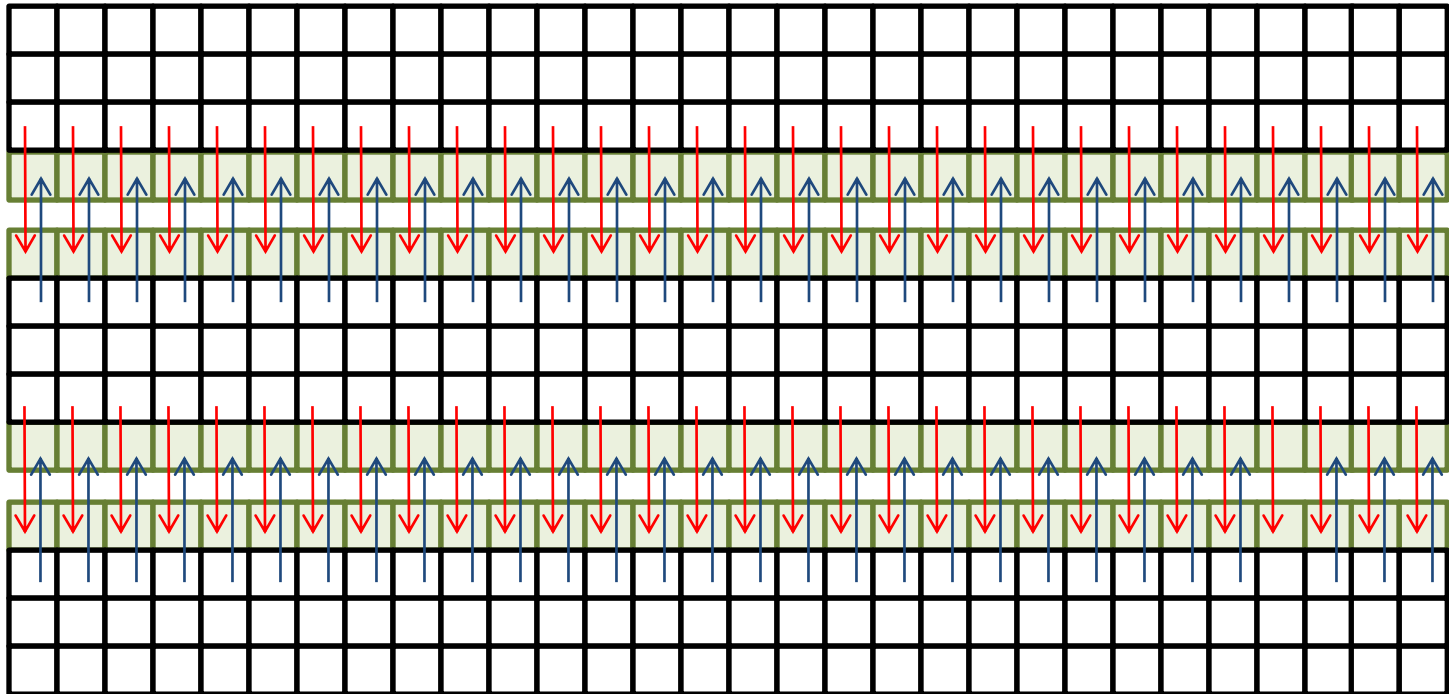


- Horizontal decomposition



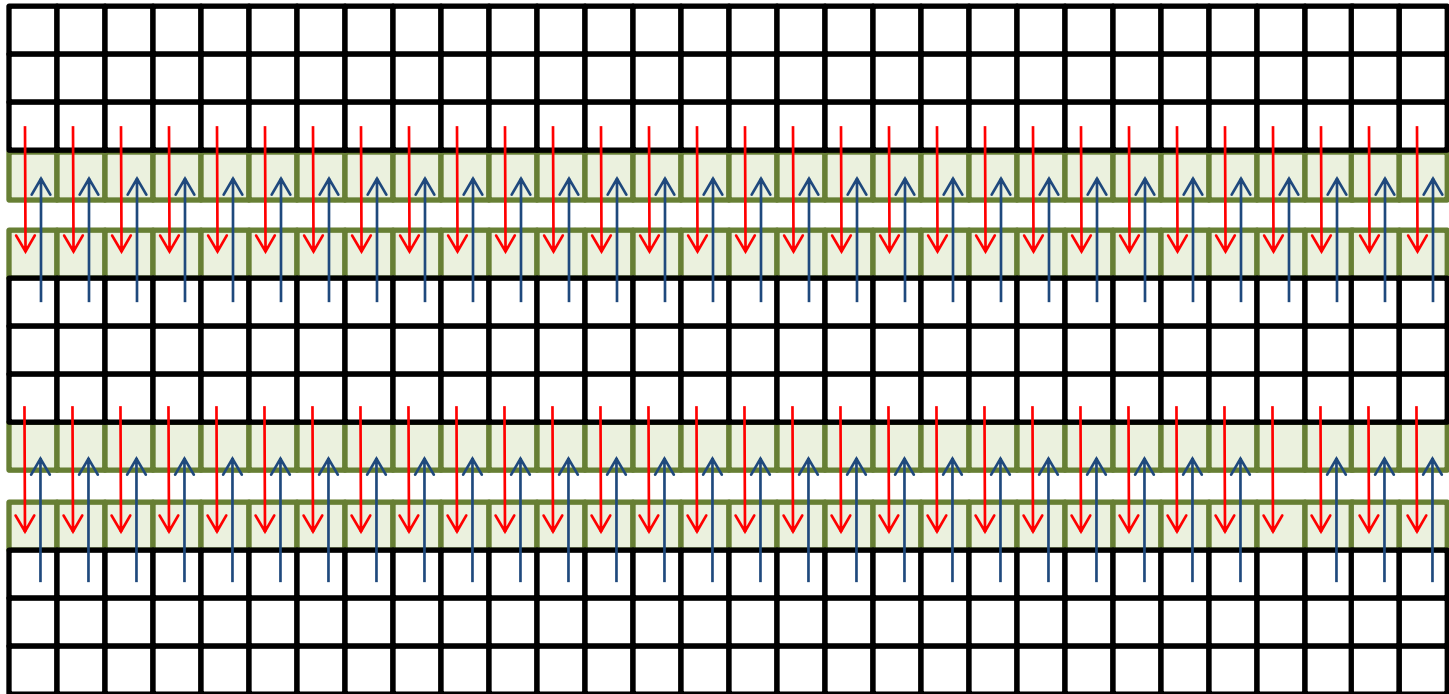
DD example: 2D matrix stencil

- Horizontal decomposition
- Ghost cells exchange



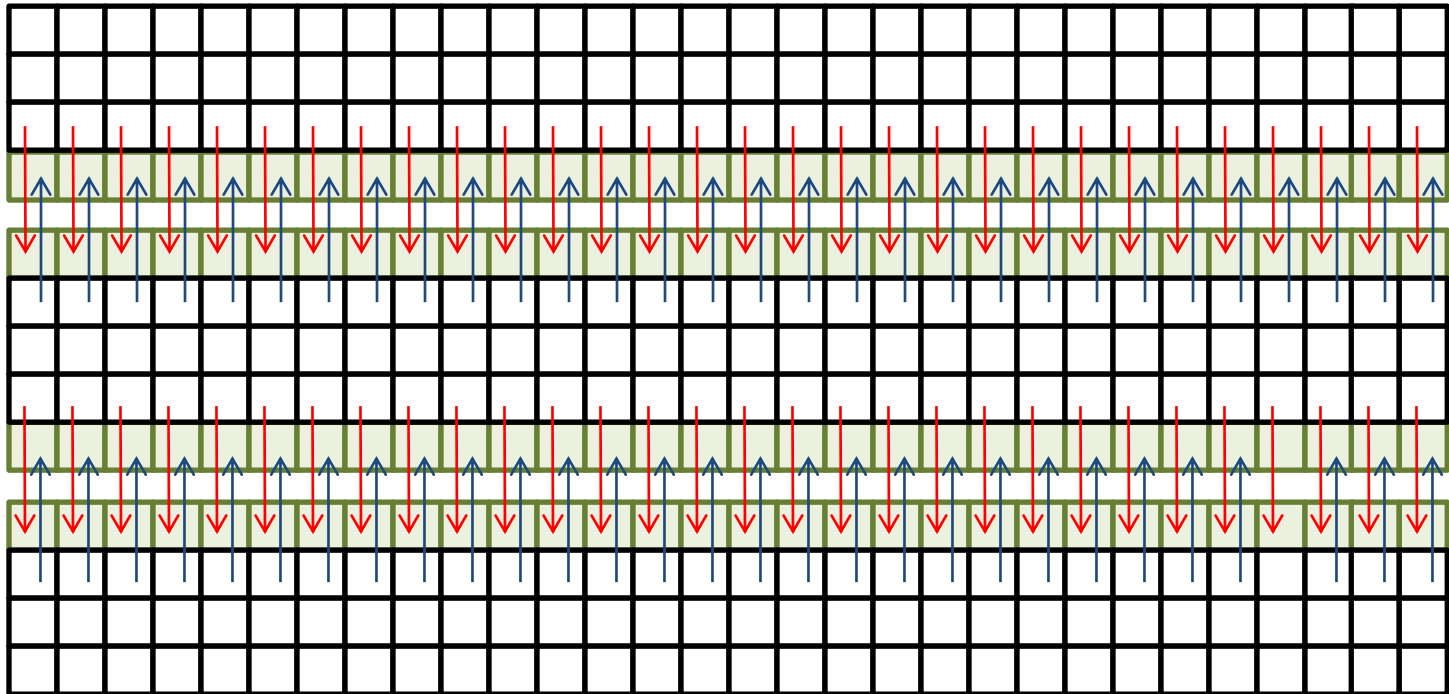
DD example: 2D matrix stencil

- Horizontal decomposition
- Ghost cells exchange
 - `send(&a[1][0], 30, MPI_INT, ...)`
 - `send(&a[H/3][1], 30, MPI_INT, ...)`



DD example: 2D matrix stencil

- Horizontal decomposition
- Ghost cells exchange: **2 sends**
 - `send(&a[1][0], 30, MPI_INT, ...)`
 - `send(&a[H/3][1], 30, MPI_INT, ...)`



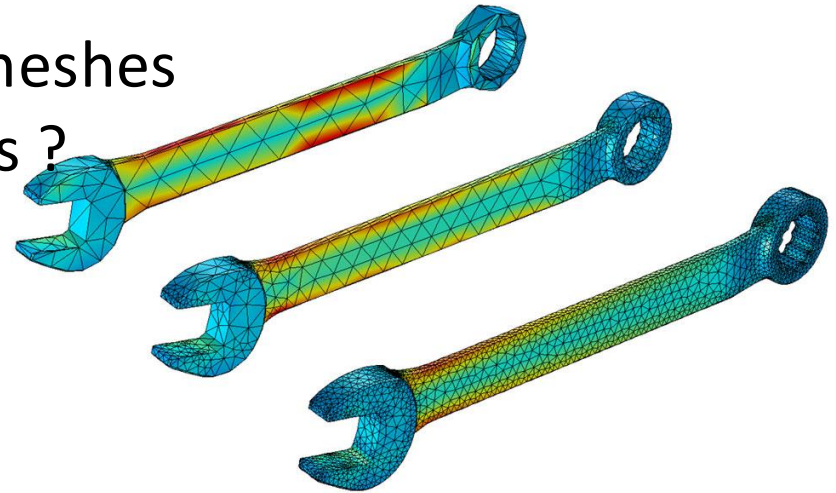
Domain decomposition



- Domain decomposition performance depends on several factors
 - Number of messages
 - Size of messages
 - Total size of exchanged bytes
- Domain decomposition is easy to do and visualize on structured meshes (multi-dimensional arrays)
- How to do it in unstructured meshes ?

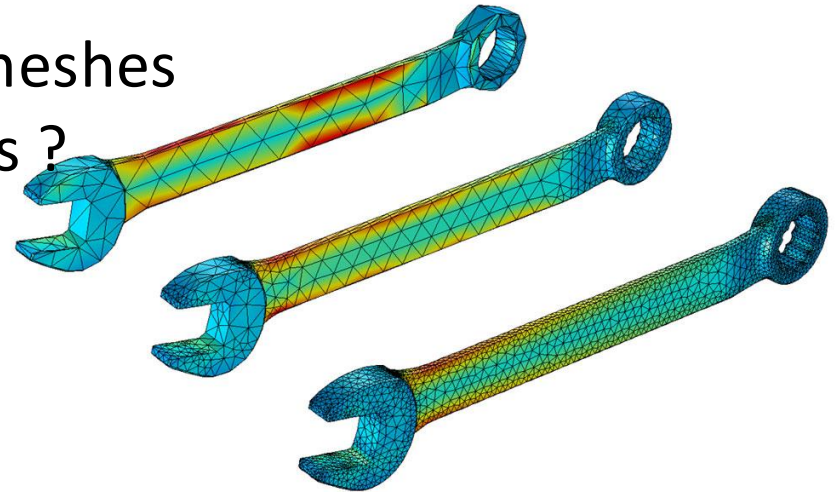
Domain decomposition

- How to do it in unstructured meshes
- What are unstructured meshes ?
 - Adaptive-mesh refinement

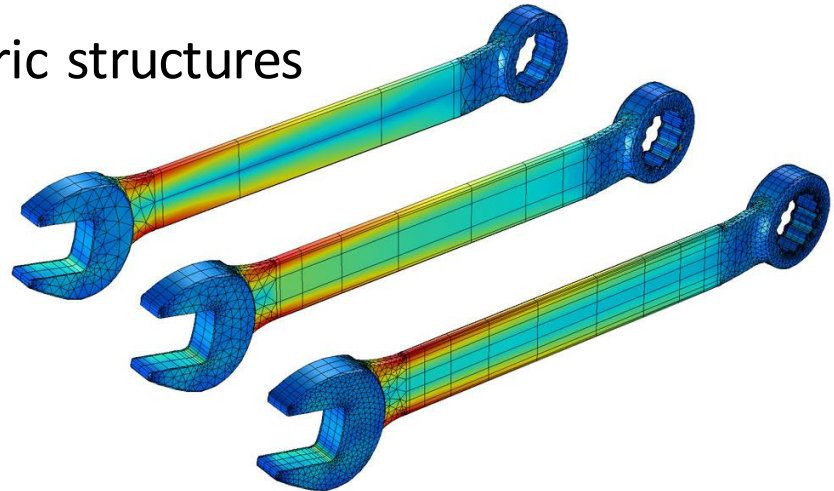


Domain decomposition

- How to do it in unstructured meshes
- What are unstructured meshes?
 - Adaptive-mesh refinement



- Different types of polyhedric structures

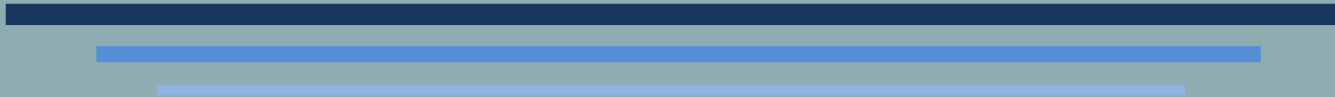


Domain decomposition



- How to do it in unstructured meshes
- Automatic partitionners exist
 - ParMETIS : <http://glaros.dtc.umn.edu/gkhome/metis/parmetis/overview>
 - Scotch : <https://gitlab.inria.fr/scotch/scotch>
 - ...
- They can be
 - Monocriterion: homogeneous partition size, homogeneous neighborhood size, homogeneous message size...
 - Multicriteria: combining multiple criteria with priority on those criteria

SOME HIGH SPEED INTERCONNECT



Used in top500 supercomputers

Mellanox Infiniband



- Bidirectional Bandwidth:
 - EDR: 200 Gb/s ,
 - HDR: 400 Gb/s
- Full CPU Offload
 - Kernel bypass: user level application get direct access to hardware
- Memory exposed to remote node access
 - RDMA-read and RDMA-write
 - Atomic operations
- Topologies:
 - Fat-tree, dragonfly+

Cray Slingshot



- Bidirectional Bandwidth: 400 Gb/s
- Fast Memory Access
 - Minimize overhead for 8-64 byte operations
 - Fast path for single word put and get
- Collective support
 - Latency optimization for their most important cases
 - Integer and floating point add
 - Max/min, compare and swap, bit operations
- SLingshot dragonfly topology
 - Average hop count flat regardless of system size
 - Up to 2 hops in source group, 1 hop to dest., up to 2 hops in dest. Group
 - Electrical in groups, optical between groups

Intel Omnipath



- Bidirectional Bandwidth: 100 Gb/s
- Designed for the needs of HPC and RDMA IO
- Data integrity
 - CRC 14bits on data path at link level
 - Several CRCs for Packet integrity
- Congestion management
 - Distributed switch based adaptive routing
 - Dispersal routing
 - Explicit congestion notification protocol
- Topology
 - Fat-tree

Fujitsu TofuD

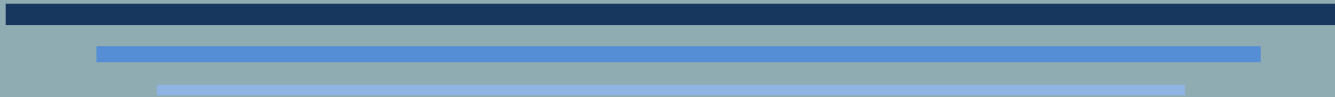


- Bidirectional Bandwidth: 108.8 Gb/s
- Cache injection
 - Tofu can inject received data into L2 cache directly
- Session mode control queue
 - Offloading a collective communication of long messages
 - Command execution may be delayed until reception of a Put
- RDMA Atomic Read-Modify-Write
 - Atomically read, modify and write back remote data
 - Typical operations: compare and swap, fetch and add
 - Mutual atomicity with processor
- Topology
 - 6D-Torus

Atos/BULL BXI

- 
- 
- Bidirectionnal Bandwidth BXI v2 : 400 Gb/s
 - HW implementation of Portals4 comm. primitives
 - NIC : handle async progress, accelerate MPI p2p matching, and offer fast path for MPI one-sided
 - OS and application bypass
 - Applications issue commands to the NIC, avoiding kernel calls
 - Data reliability
 - CRC 32bits added to each message
 - Protocol to retransmit lost or corrupted messages
 - Topologies
 - Fat-tree, torus, flattened-dragonfly

SOME HIGH-SPEED NETWORK TOPOLOGIES



Network topologies



- In a supercomputer, the compute nodes are connected through a high-speed network
- Each node doesn't have a direct link to all other nodes it needs to communicate with
 - Too expensive
- Network design must be optimized for a good trade-off cost / efficiency

Network topologies

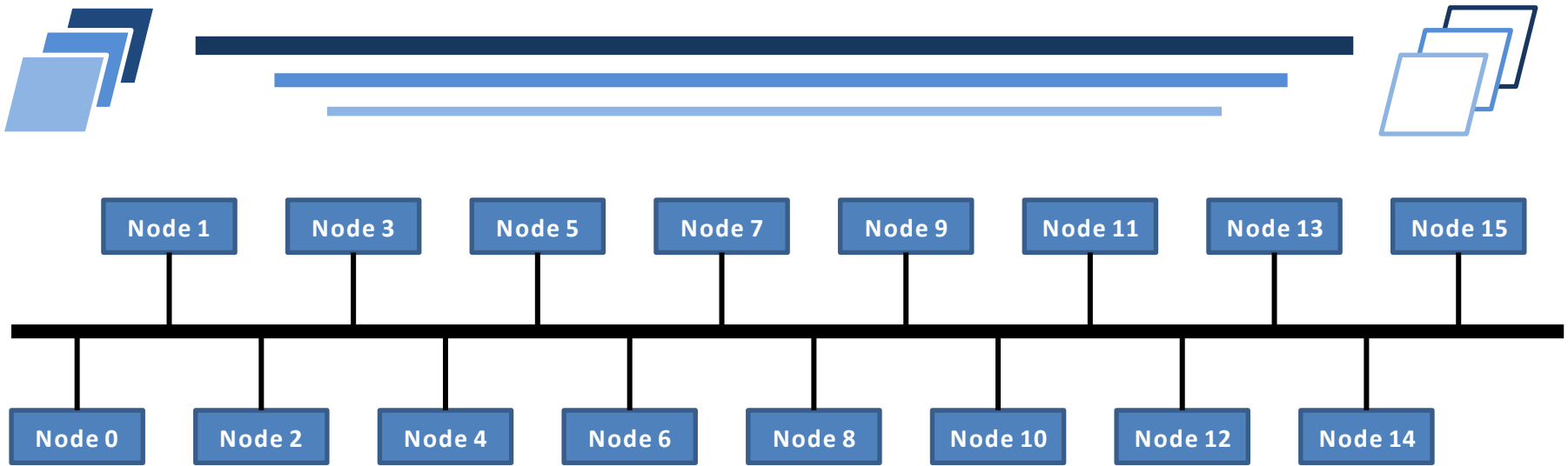
- Two **performance criteria** for a network connecting **n** nodes:
 - Mean number of links to connect 2 nodes : n_L
 - Bisection Bandwidth (bandwidth between the two halves of the whole network): B_b

Network topologies



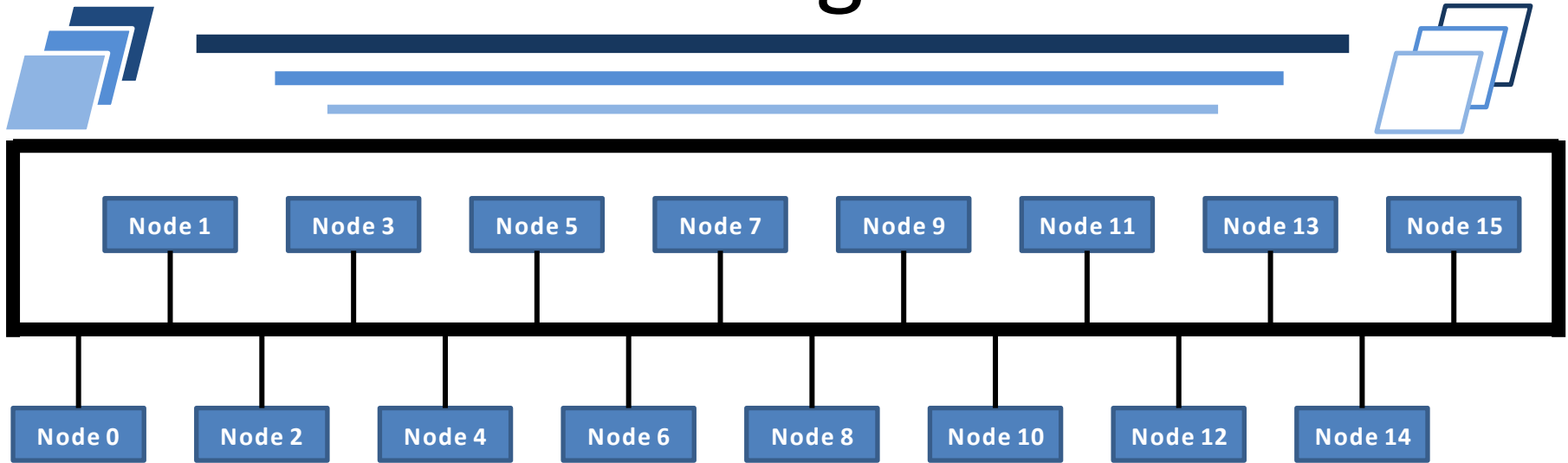
- Numerous different network topologies exist
 - Bus, Ring
 - Grid, Torus
 - Trees
 - Hypercube
 - Butterfly, Dragonfly
 - ...

Bus



- All nodes are connected through a bus
- Simple
- Cost effective for a small number of nodes
- High contention
 - Not scalable for high number of nodes
- $n_L \sim n/3$
- $B_b \sim 1$

Ring



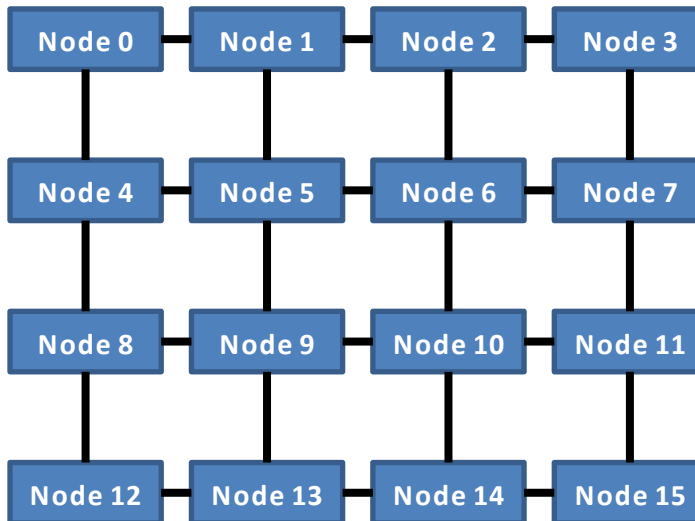
- Generalization of Bus topology
- All nodes are connected through a bus
- Both ends of the bus are connected
- Better contention
 - Still not scalable for high number of nodes
- $n_L \sim n/4$
- $B_b \geq 2$

Characteristics



- Low cost
- Low adaptability
- Low bandwidth
- Low scalability
- High latency
- → Not used in HPC

Mesh topology



■ $n_L \sim \sqrt{n}$

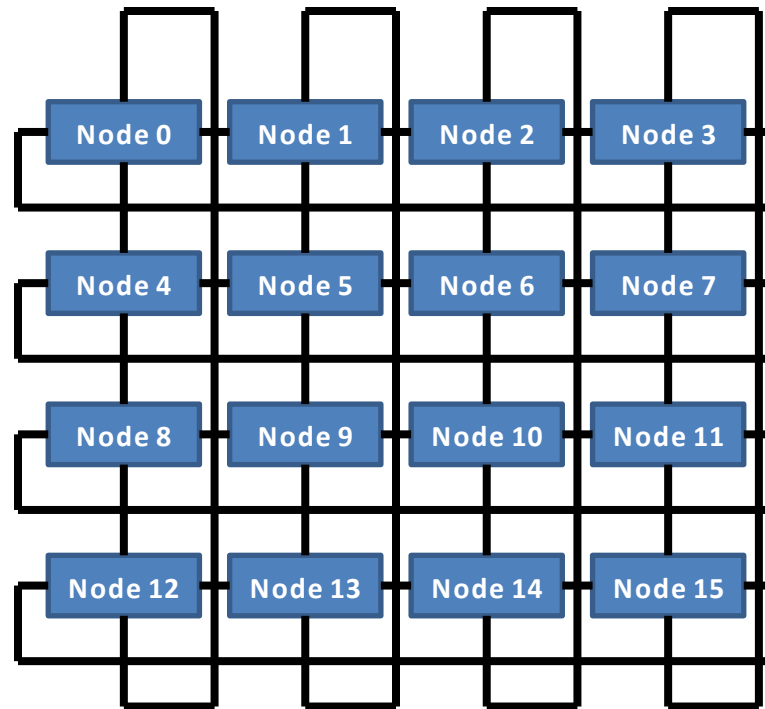
■ $B_b = \sqrt{n}$

Characteristics



- Limitate number of neighbors:
 - Lower cost
 - Lower latency
- Easy to design
- Each link is the same
- Irregular on the borders of the topology
 - Codes need to adapt to preserve communications locality
- Require an optimized policy from the ressource manager/allocator
 - Groups the nodes of the same application

Torus



■ $n_L \sim \sqrt{n}/2$

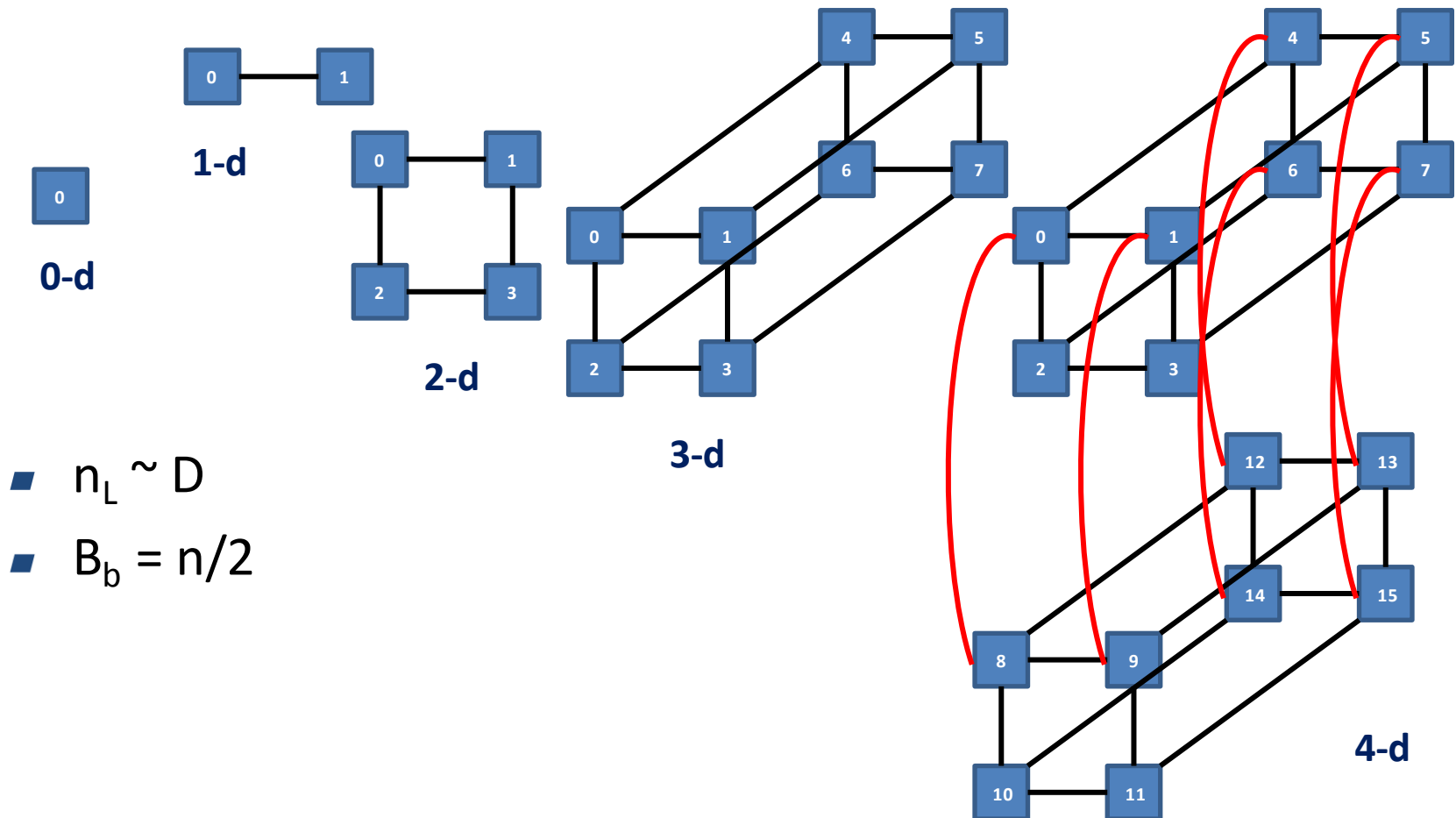
■ $B_b = 2\sqrt{n}$

Characteristics



- Nearly the same as the Grid
- In addition, not irregular on the « borders »...
- ... except the length of the connecting links
- Hard to design and to install
- Used in :
 - Fugaku Tofu-D(#2 top500)
 - BG/L, BG/P

Hypercube with D-dimensions



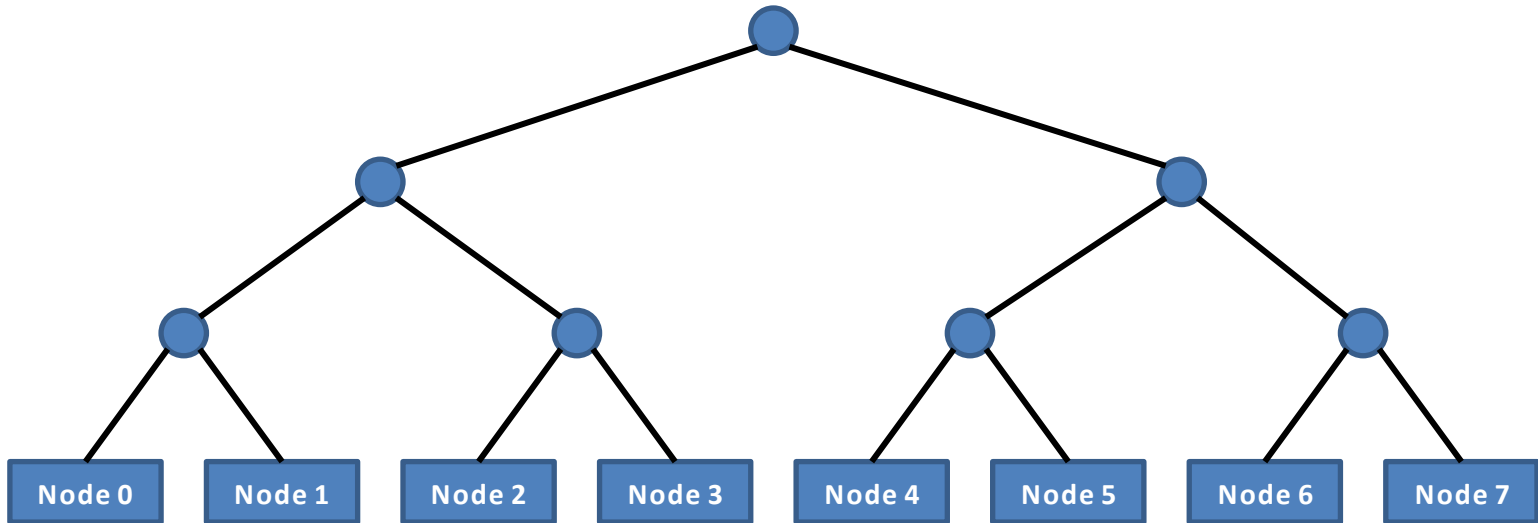
- $n_L \sim D$
- $B_b = n/2$

Characteristics



- Limitate number of neighbors:
 - Lower cost
 - Lower latency
- Irregular on the borders of the topology
 - Codes need to adapt to preserve communications locality
- Require an optimized policy from the ressource manager/allocator
 - Groups the nodes of the same application
- Harder to design on a 2D or 3D chip
- Used in:
 - SGI

Tree



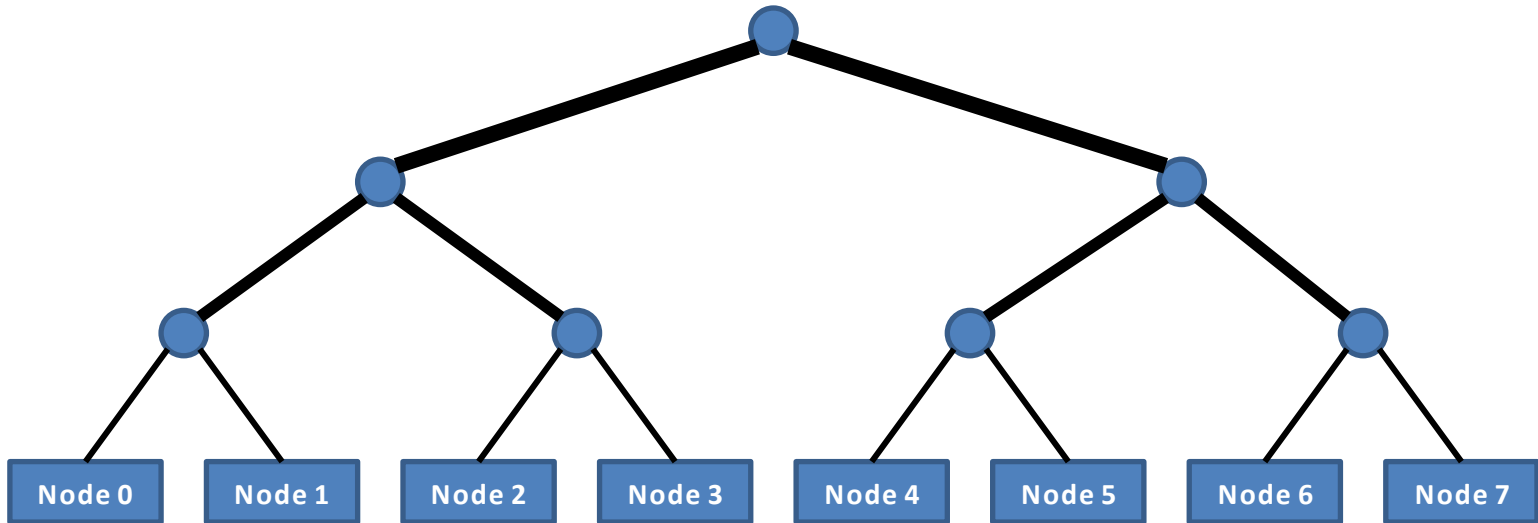
- $n_L \sim \log(n)$
- $B_b = 1$

Characteristics



- Average cost
- Low latency
 - Few hops between nodes
- Easy to design
- Very good for local traffic...
- ... but the root can be overloaded if there are lots of long-distance messages
 - congestion

Fat-tree



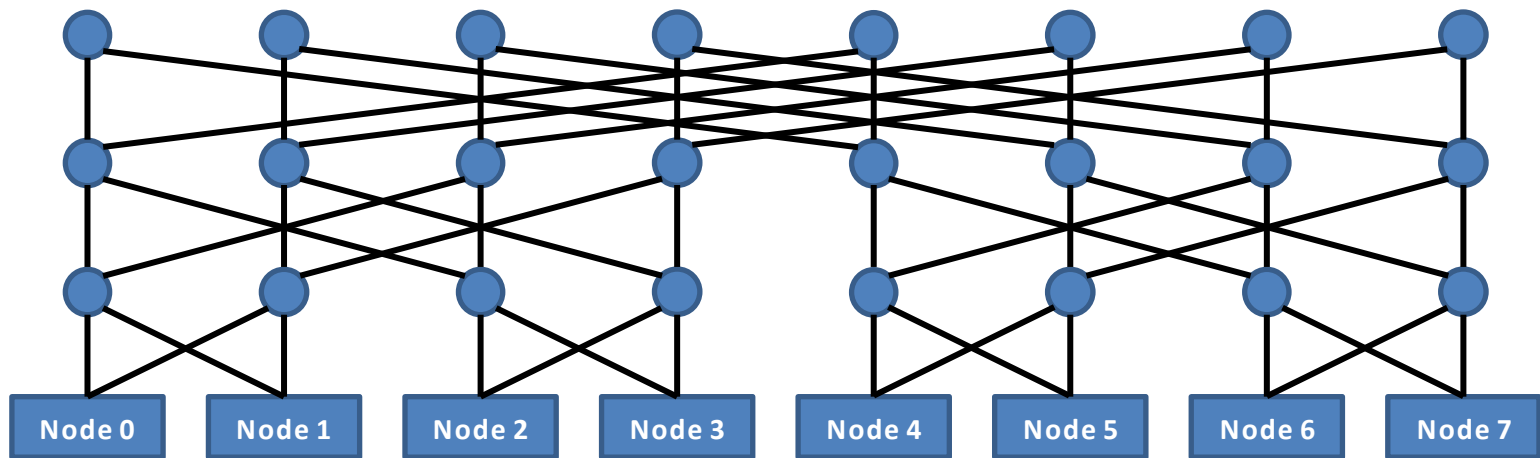
- $n_L \sim \log(n)$
- $B_b = n/2...?$

Characteristics



- High cost
- Each level has more physical links than the previous one
- → More bandwidth at the root to avoid congestion
- Greedy resource management policy
 - Often used because it reduces the user waiting time
- Reality less ideal than theory
 - Network switches are very loaded
- Used in
 - TERA 100, TERA 1000
 - Summit, Sierra, Selene infiniband

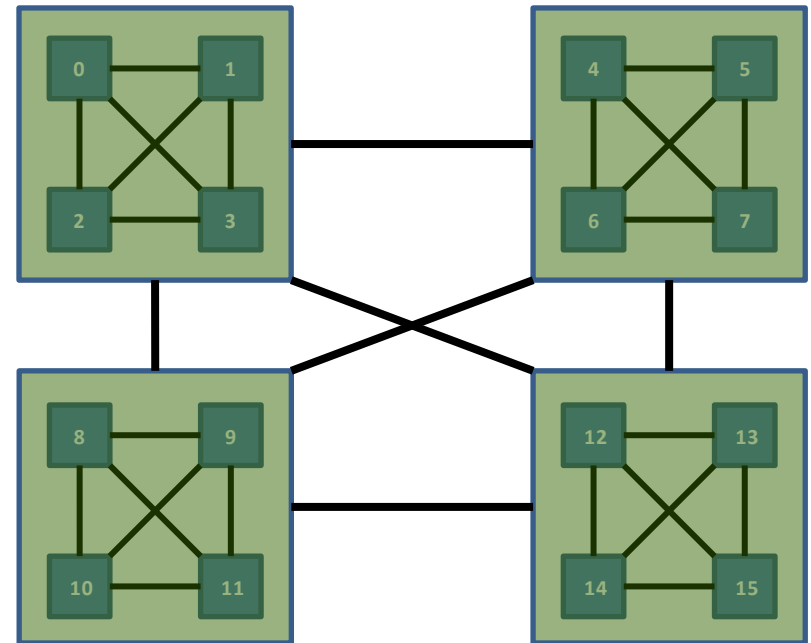
Other topologies: Butterfly



- $n_L \sim \log_k(n+1)$ (ici $\log_2(n+1)$)
- $B_b = n/2$

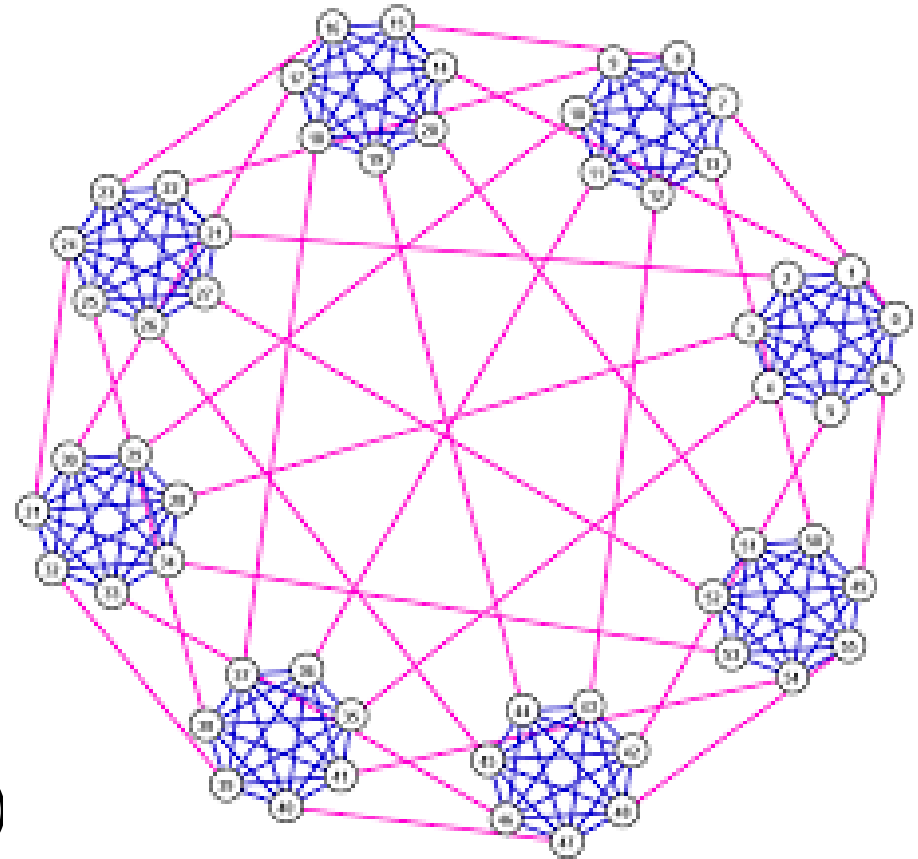
Other topologies: dragonfly

- Full-connected graph of full-connected graphs
- $n_L \sim 3$
- $B_b = (n \sqrt{n})/4$
- Used in:
 - Frontier (#1 top500) and first exascale machine
 - Lumi (# 3), Adastral (# 11)



Other topologies: dragonfly

- Full-connected graph of full-connected graphs
- $n_L \sim 3$
- $B_b = (n \sqrt{n})/4$
- Used in:
 - Frontier (#1 top500) and first exascale machine
 - Lumi (# 3), Adastral (# 11)



General remarks



- Network topology can impact which algorithm can be the best
 - Most MPI implementations don't consider hardware topology in the algorithm choice (too costly)
- Network topology also impacts resource allocation
 - Most MPI implementations also don't consider how resources (nodes) have been allocated to the current job
- Moreover, how ranks are given to MPI processes can depend on:
 - Network topology
 - Resource allocation

Resource manager (1)



- Slurm: Simple Linux Utility for Resource Management
 - [open source](#)
 - Allows to create sub-clusters (partitions) in Linux with some fault-tolerance
 - ip-failover
 - Also schedules the job on the physical resources of the supercomputer.
- It can be used on clusters of various sizes, from two to hundred thousand nodes.

Resource manager (2)



- Slurm doesn't require any modification of the Linux kernel to work and is autonomous
- As resource manager on clusters, Slurm has three main functionalities:
 - Allocate and give access to resources for all users, for a limited amount of time so they can run their jobs ;
 - Provide mechanisms to launch, execute and follow the status of the jobs on the allocated resources ;
 - Schedule the access to resources and the jobs by managing a waiting list.

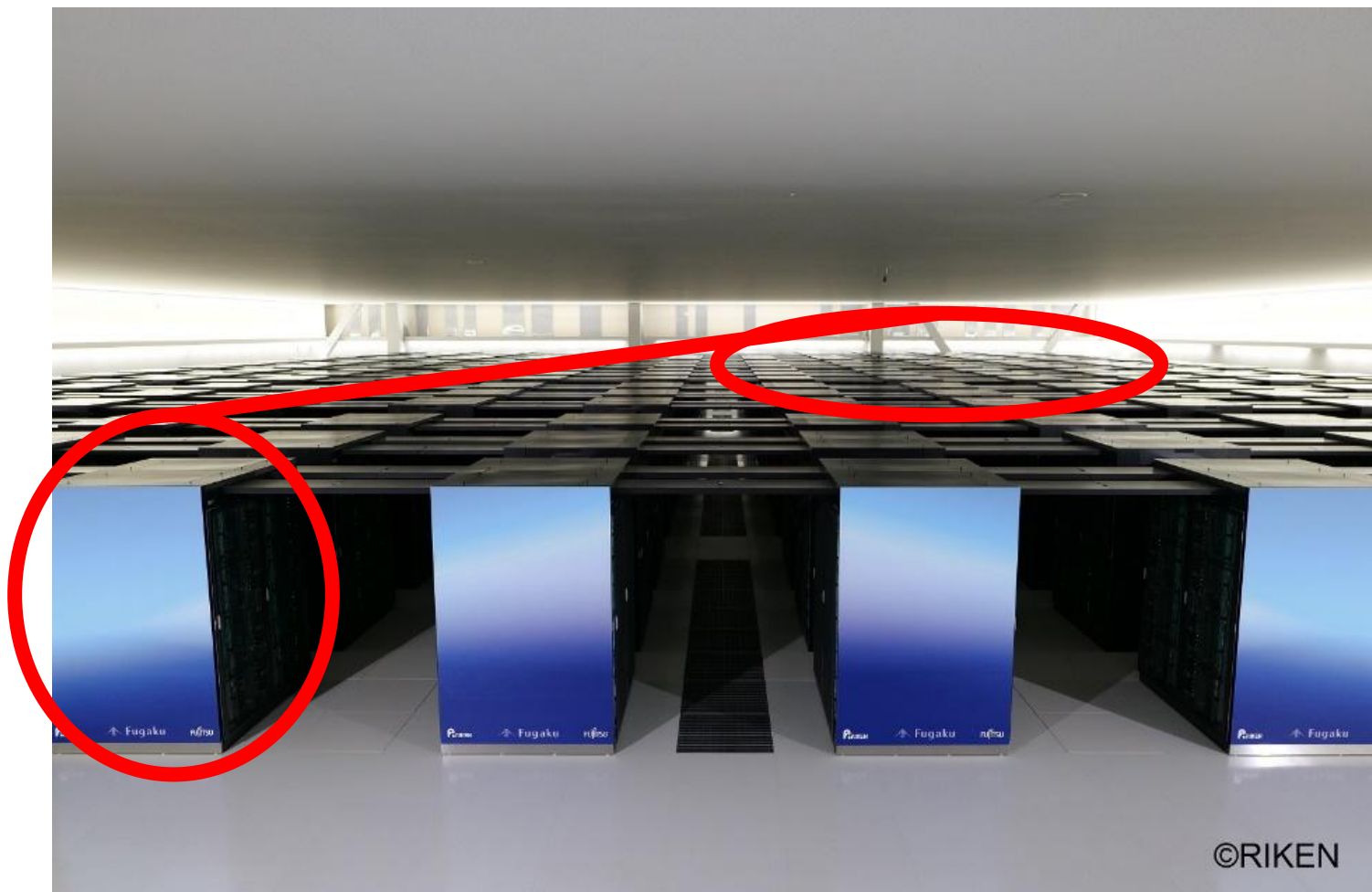
Resource manager (3)



- The resource manager do its best regarding the scheduling of jobs and the demands for resources
- There are no guarantees that the provides resources (nodes) are adjacent and will benefit from the underlying topology
 - You get what is available
 - Not possible to deallocate/migrate running jobs to free resources and have a better locality
- It is the user's responsibility to detect and adapt to the actual topology of allocated resources

Resource manager (3)

- What if you end up with this allocation ?

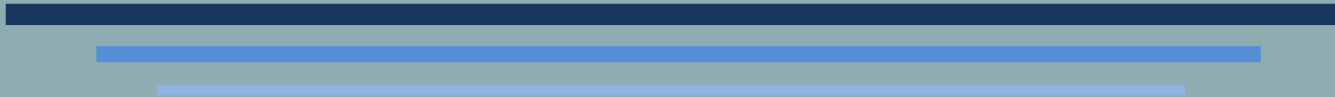


Hardware topology



- Hardware topology and resource allocation can be source of performance loss
- May be necessary to take these into account to reach the best performances possible
- Latest version of MPI standard (4.0) includes a new feature to take into account underlying hardware topology

MPI 4.0 HARDWARE-TOPOLOGY FEATURE



Communicator topological splitting



- In previous MPI standard version, no hw topology mention
 - Except for virtual topologies
 - Possible to reorder rank numbers to better map hw topology
 - Very seldom use
- Add new comm-split type
 - Split a communicator into subcommunicators following the hardware topology
- 2 mechanisms:
 - Unguided: split the current communicator into subcommunicators regrouping the MPI processes sharing the next hw topology level
 - Guided: split the current communicator into subcommunicators regrouping the MPI processes sharing the required hw topology level

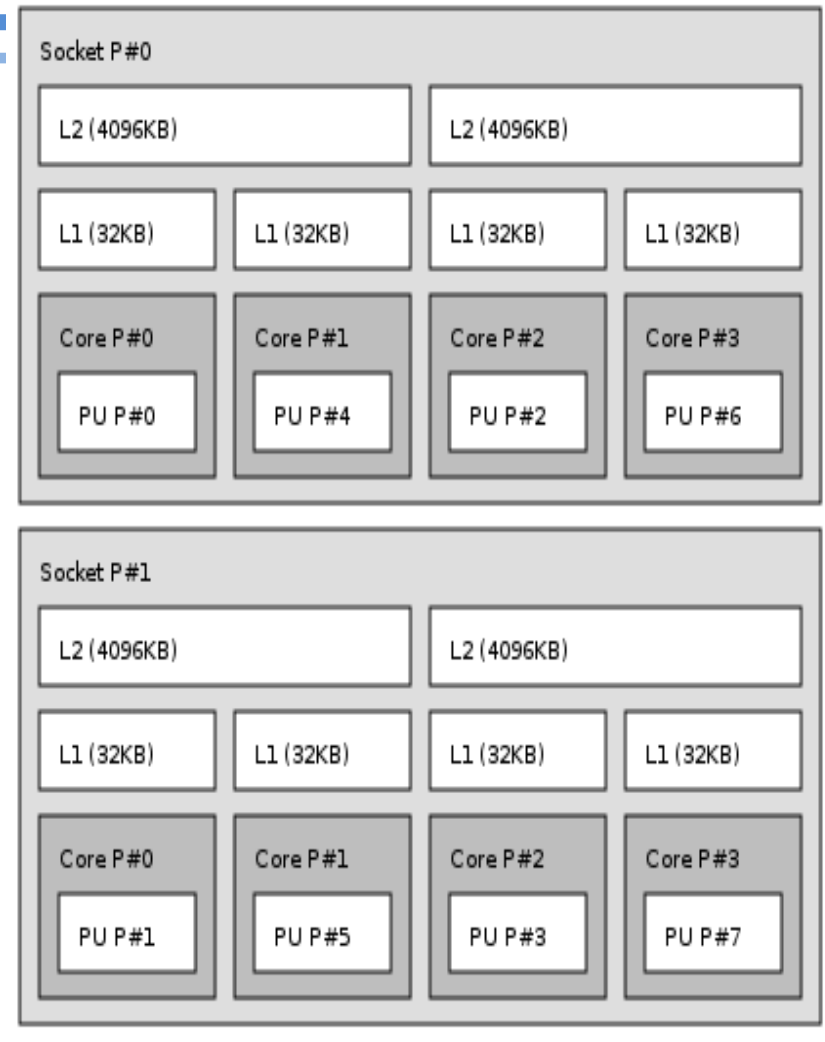
Communicator topological splitting

■ Unguided mode example

```
hwcomm[level_num] = MPI_COMM_WORLD;

while((hwcomm[level_num] != MPI_COMM_NULL)
      && (level_num < MAX_NUM_LEVELS-1)){
    MPI_Comm_rank(hwcomm[level_num], &rank);
    MPI_Comm_split_type(hwcomm[level_num],
                        MPI_COMM_TYPE_HW_UNGUIDED,
                        rank,
                        MPI_INFO_NULL,
                        &hwcomm[level_num+1]);
    level_num++;
}
```

Machine (16GB)

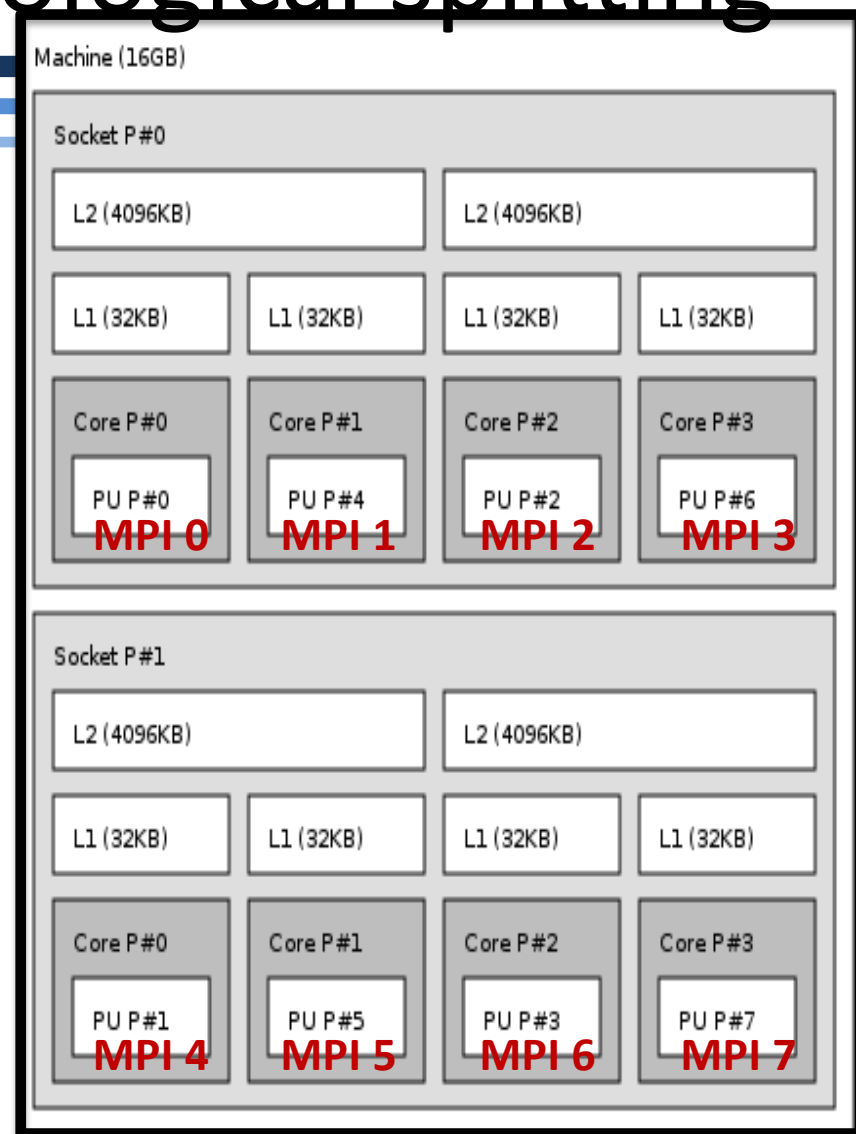


Communicator topological splitting

- Unguided mode example:
 - `MPI_Comm_world={0,1,2,3,4,5,6,7}`

```
hwcomm[level_num] = MPI_COMM_WORLD;
```

```
while((hwcomm[level_num] != MPI_COMM_NULL)
      && (level_num < MAX_NUM_LEVELS-1)){
    MPI_Comm_rank(hwcomm[level_num], &rank);
    MPI_Comm_split_type(hwcomm[level_num],
                        MPI_COMM_TYPE_HW_UNGUIDED,
                        rank,
                        MPI_INFO_NULL,
                        &hwcomm[level_num+1]);
    level_num++;
}
```

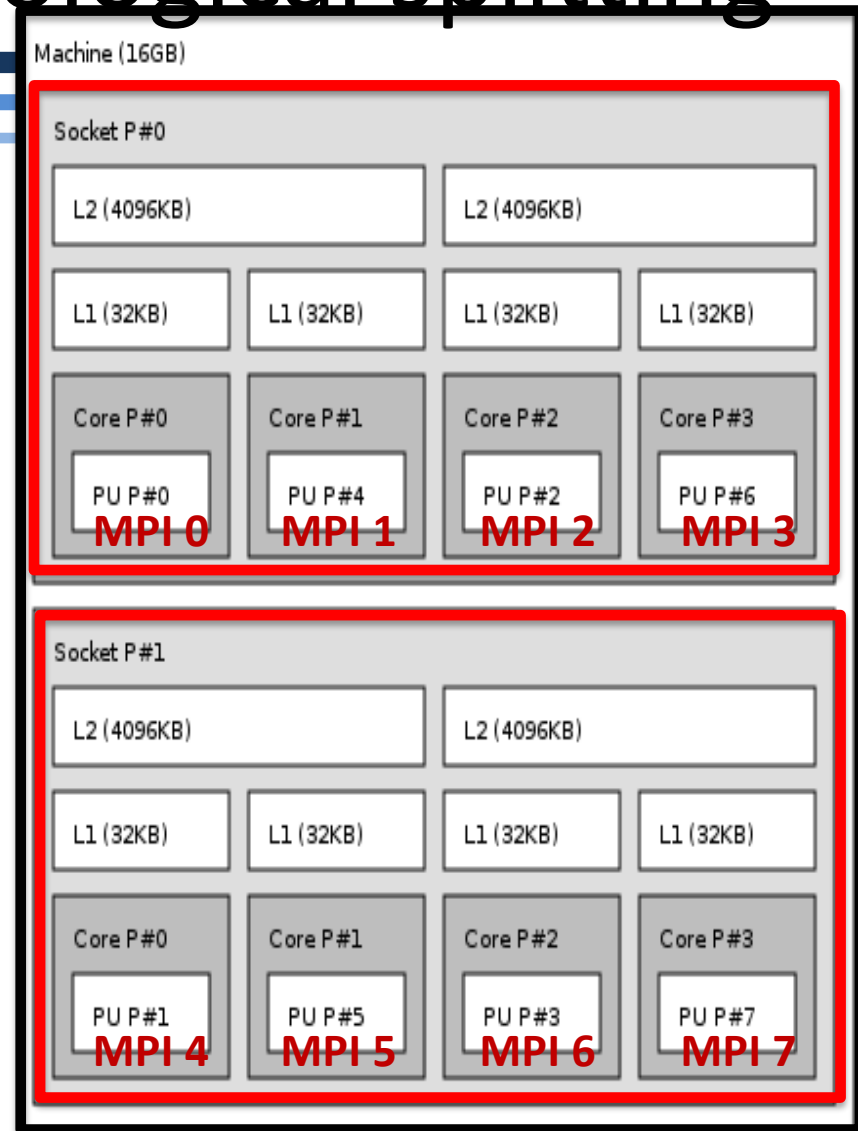


Communicator topological splitting

- Unguided mode example:
 - `MPI_Comm_world={0,1,2,3,4,5,6,7}`
 - `Comm1={0,1,2,3}` |
`Comm2={4,5,6,7}`

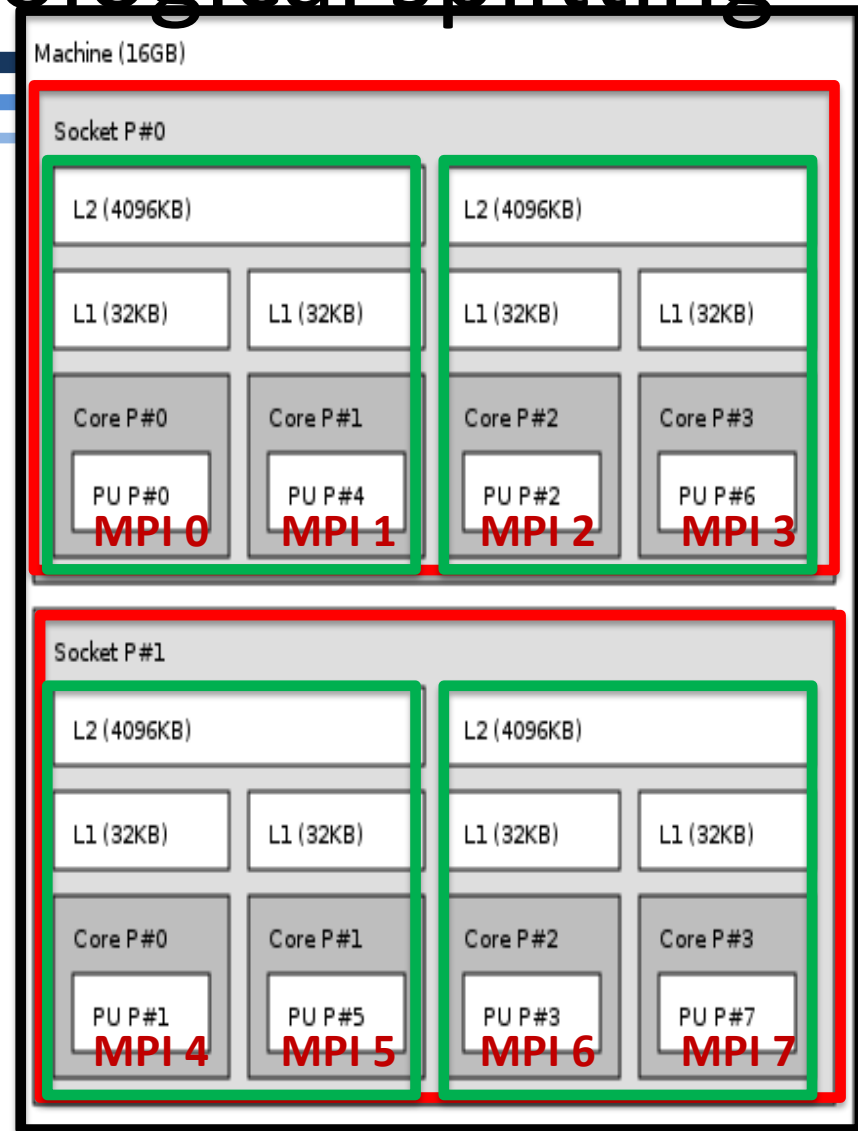
```
hwcomm[level_num] = MPI_COMM_WORLD;
```

```
while((hwcomm[level_num] != MPI_COMM_NULL)
      && (level_num < MAX_NUM_LEVELS-1)){
    MPI_Comm_rank(hwcomm[level_num], &rank);
    MPI_Comm_split_type(hwcomm[level_num],
                        MPI_COMM_TYPE_HW_UNGUIDED,
                        rank,
                        MPI_INFO_NULL,
                        &hwcomm[level_num+1]);
    level_num++;
}
```



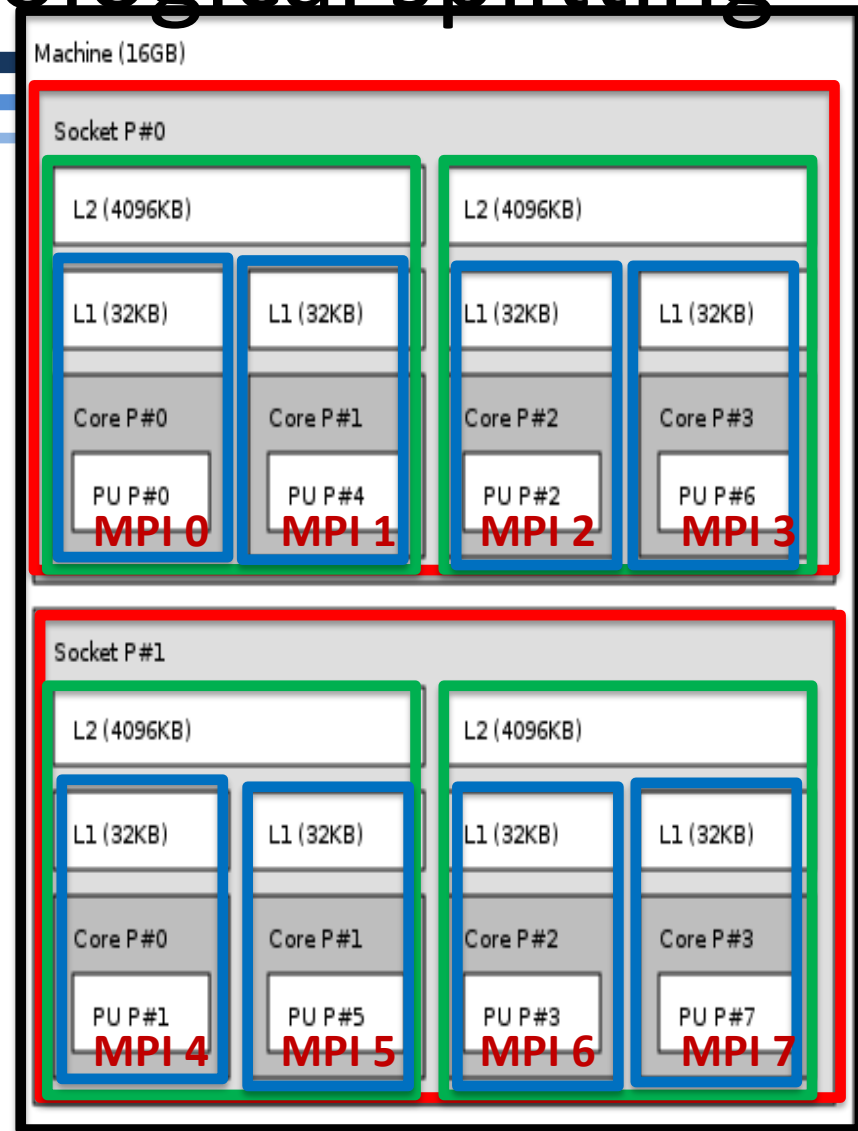
Communicator topological splitting

- Unguided mode example:
 - `MPI_Comm_world={0,1,2,3,4,5,6,7}`
 - `Comm1={0,1,2,3}` | `Comm2={4,5,6,7}`
 - `Comm3={0,1}` | `Comm4={2,3}` | `Comm5={4,5}` | `Comm6={6,7}`



Communicator topological splitting

- Unguided mode example:
 - `MPI_Comm_world={0,1,2,3,4,5,6,7}`
 - `Comm1={0,1,2,3}` | `Comm2={4,5,6,7}`
 - `Comm3={0,1}` | `Comm4={2,3}` | `Comm5={4,5}` | `Comm6={6,7}`
 - `Comm7={0}` | `Comm8={1}` | `Comm9={2}` | `Comm10={3}` | `Comm11={4}` | `Comm12={5}` | `Comm13={6}` | `Comm14={7}`

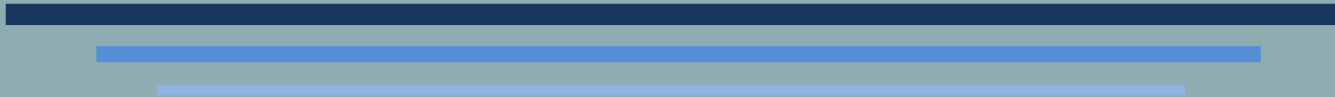


Communicator topological splitting



- Levels supported are implementation defined
 - It is correct to support no level at all
- Most implementations today rely on hwloc to provide hardware information
 - Unfortunately, it only provides hardware topology inside a node
 - No network information
- Tentative Netloc tool is too difficult to implement and is now dead
 - But local efforts are done by implementation to have some kind of network knowledge

MPI FORUM



MPI Forum

- The MPI Forum is the standardisation forum for Message Passing Interface (MPI)
- The MPI Forum has several roles:
 - Conception of the MPI standard
 - Correction of text errors in the standard
 - Discussion of new directions for the standard
 - Suggestion of modifications to the standard
 - To add new features
 - To deprecate features
 - To improve/maintain the coherency of the document
 - To improve/maintain quality and readability of the document
- The MPI Forum represents MPI in the scientific community
 - ...and should be the link between the needs of scientists and the redaction (and implementation) of the MPI standard.



MPI Forum organization



- The MPI Forum is composed of several scientists of numerous organizations
 - National labs, Microsoft, Intel, Nvidia, IBM, CISCO, Mellanox, HLRS, JSC, CEA, Paratools, Universities (Tennessee, Urbana-Champaign, Bordeaux, EPCC, ...) ...
- Separated in two overlapping sub-committee:
 - Chapter committee, focusing on the quality and coherency of the document
 - Each chapter in the MPI Standard has its own president.
 - Work Groups, discussing the evolution of the standard (new ideas, corrections, deprecations, ...)
 - Each Work Group focus on its own subject (IO, RMA, collectives, hybrid, tools,...)
 - Bi-weekly or monthly phone conf to discuss subjects on the Work Groups

MPI Forum meetings



■ MPI Forum Meetings

- Face-to-face meetings every three months
- Three in the US, and one with EuroMPI
- Work Group discussions, presentations of new idea, modifications or results
- Vote to include new features in the next version of the standard

■ Voting rules

- Voting right if the organization has been present 1 time in the two previous meeting
 - No membership fee
 - Registration fee for the attended meetings
- Quorum must be reached: 2/3 of organizations with voting right
- API modification must be submitted in formatted version 2 weeks before the meeting
 - If text has been touched, no-no votes is required
- For errata correction, only one vote is necessary
- For big modification, one formal reading and two votes are required
 - Each step at a different Meeting

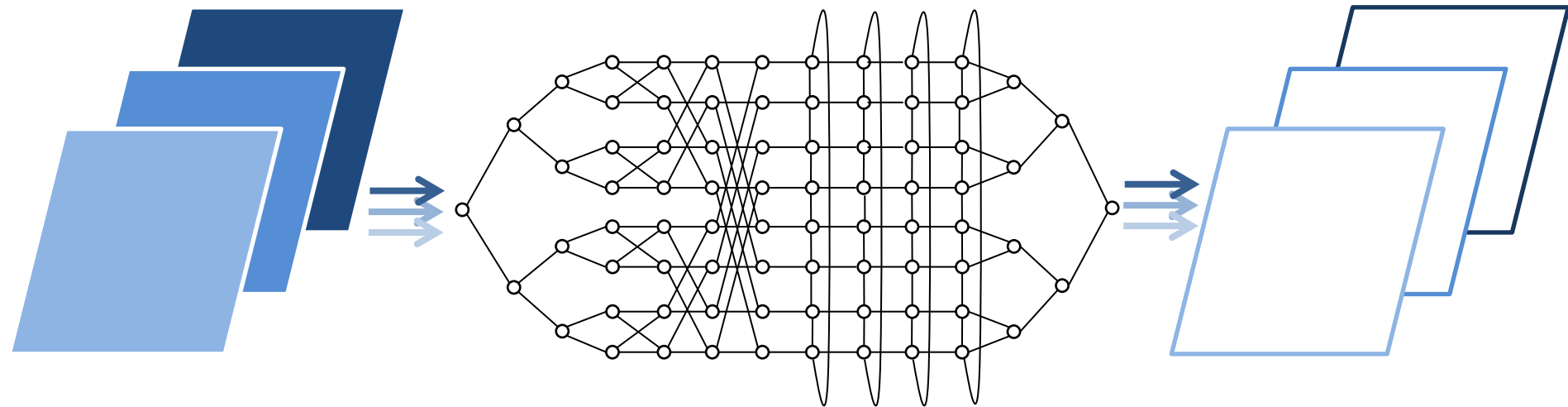
MPI Forum



- MPI Forum information can be found on the website:
<https://www.mpi-forum.org/>
- Weekly calls on wednesdays (virtual meetings) to discuss specific points
- Each WG has its own concall meetings
- Completely free to attend!

Programmation Parallèle

MPI: Message Passing Interface



FIN DU COURS